HackPit

Full Capability Assessment & Agreed Plan

Date: 26 July 2026

Scope: backend · frontend · pipeline · docker · docs · knowledge base

Measured against: OSCP · PNPT · eCPPT · HTB CPTS · CRTP · HTB/CTF · bug bounty · client pentests

What is re-locked, unchanged

KB exploitation-writeup corpus — the case-based material 2.7 keys into (follow-on, now landed)

Verification

Status

Build #9 — the Windows/AD path, driven live against a real domain (2026-07-31)

Build #10 — the opt-in C2 exposure, and an honest NOT-RUN (2026-07-31)

Build #11 — the launcher, and operator identity (2026-07-31)

A dozen surfaces were built and then invisible

The status rail answers a question the UI could not

The report could not identify its own author

A mistake worth recording

What is locked

HackPit — Capability Assessment & Build Log

Date: 2026-07-26 **Scope:** every backend module, the frontend, the pipeline, the docker stack, the docs, and the live KB **Targets this is measured against:** OSCP · PNPT · eCPPT · HTB CPTS · CRTP · HTB boxes/CTF · bug bounty · real-world client pentests.

Status note (updated). This began as an assessment (Part I) and an agreed plan (Part II). **All four phases of that plan have since been built**, and a fifth followed — the execution substrate (Phase 1), the state model (Phase 2), the drive-speed gaps (Phase 3), the content-and-goal-specific work (Phase 4: KB Route-B ingest, the evasion/OPSEC channel, the HTTP repeater, pivot/tunnel routing, exam-mode report templates), and **Phase 5**, which closed the two things Part I had left standing: a real PTY (added as a *second* surface, so the auditable one survives) and the recommended KB enrichment (PortSwigger Academy, HackTricks Cloud, the HTB slice, and a CVE→exploit index). Part I has been trimmed to what genuinely remains; the completed blockers, decisions and phases live in **Part III — Build Log**.

---

0. Verdict

The **architecture is genuinely strong** — the safety engineering and the LLM-grounding discipline are better than most commercial tooling.

The original problem this assessment found was *reach*: nothing in HackPit was fake or stubbed, but the container it executed into contained seven programs, so it could not finish a real HTB box, touch a real AD domain, run a bug-bounty recon sweep, or complete an OSCP-style engagement. **That execution substrate has now been rebuilt (Phase 1), the loop reasons over structured state instead of stdout tails (Phase 2), the drive-speed gaps are closed (Phase 3), the content/goal work is done (Phase 4), and the interaction and knowledge gaps Part I left standing are closed (Phase 5).** See Part III.

Since then **Phase 5** closed the PTY gap (a real terminal as a *second* surface, keeping the auditable one) and finished the KB enrichment the assessment recommended, and the **Windows execution backend** (a WinRM driver) has since made the CRTP toolset, the OSCP AD set and real internal pentests *executable* — and made the existing AD attack-path graph run **live** instead of on synthetic data. What remains is a short list of *deliberately deferred* items — a VPS for blind callbacks, and the last few thin KB categories — each deferred for a stated reason, not left undone. The Windows/CRTP execution target is **no longer among them**: it is closed by the WinRM driver against an external VM you run (see the Windows execution backend in Part III and D9). Risk-tiered approval is also gone — **decided against** (see D5). See §1 and Part II's "Explicitly deferred".

---

1. Remaining gaps

The blocker sections for everything the five phases fixed have been removed (see Part III). What is left is genuinely small.

1.1 The PTY gap — now closed, without losing the transcript

This section previously read "no true PTY — deliberate partial", and framed it as a trade: readable logged transcripts *or* full-screen tooling. **Phase 5 rejected that trade.** The premise was wrong — the two are not alternatives if they are separate surfaces.

`:kali` is unchanged: still a persistent shell delimiting each command with a sentinel over a plain pipe, still producing the clean, escape-free, per-command transcripts reports are built from. Alongside it, `:terminal` allocates a **real PTY** in the same open sandbox, so `vim`, `top`, an interactive `msfconsole`, a raw `evil-winrm` shell and `python -c 'pty.spawn'` upgrades all render. Both are audited; both carry identical containment. See Part III, Phase 5.

1.2 The human-approval model — settled, not open

Current state: every command needs individual `approved=true`; anything the danger heuristic flags additionally needs `dangerous_ack`; `:kali` and `:terminal` are human-driven (typing *is* the approval). Phase 3 added **one-keystroke approve** (`Enter` = approve · `S` = skip · `Esc` = stop; a dangerous command never fires on `Enter` alone).

This section previously proposed **risk-tiered approval** — auto-running "passive" commands, batch-approving a plan of read-only ones — as open work. **That is now decided against** (D5). Per-command approval is the project's single load-bearing safety property in engagement mode: Wall A is down, the sandbox reaches the internet, the host and the LAN, and nothing else bounds *where* a command can go. A tier that auto-runs anything would remove the only gate on the one mode that needs it most, and it would do so by trusting a classifier — a new component, on the wrong side of the safety boundary, to decide when the human can be skipped. The convenience it buys is a keystroke that has already been optimised away.

Per-command approval is therefore **standing policy, not a deferred item**, and does not appear in the deferred list. The prerequisite work that would have enabled tiering (`_looks_like_host()`, Phase 3 step 12) was worth doing on its own merits — the scope check is trustworthy now regardless. **Unchanged and non-negotiable: never remove the gate for anything the danger heuristic flags.**

---

2. Feature-by-feature audit

Companion (KB / search / attack paths)

Component	State	Notes
KB load + serve	**Solid**	1,601 entries in memory, exclusions dropped at the door and re-filtered defensively at query time.
Hybrid search (`pipeline/search.py`)	**Solid**	BM25 (stdlib) + `nomic-embed-text` cosine, weighted RRF (lex 1.0 / vec 0.5 — correct call for a KB full of exact identifiers), tier boost, whole-query title bonus. Degrades to lexical when Ollama is down instead of 500-ing.
Attack-path composition	**Solid, genuinely novel**	Writeup-first mode + KB-first mode. Retrieval seeded per-phase so no phase starts empty. Step eligibility filtering (`is_step_eligible`) is unusually careful — excludes writeups, defensive-hardening guides, tool-install meta-docs, personal logs, grab-bags.

Component	State	Notes
Grounding / validation	**Best-in-class**	Cited `entry_id` s must resolve or the step downgrades to `ai_suggested` ; commands come from the KB entry, never the model; `target_adaptation` is dropped entirely if it names an FQDN not in the supplied facts.
`substitute_target`	**Excellent**	CIDR sentinel to avoid double-rewrite, host-position gating so `.env.example` and `<script>` survive, refuses to rewrite what it can't rewrite confidently.
`foreign_refs`	**Excellent**	Honesty marker — reports a foreign host/AD domain rather than guessing a replacement. Right call.
Channel-2 context grounding	**Good**	Writeup/methodology content as reasoning background, strictly separated from the step pool, with a leak guard over model output.
Engagement sessions	**Works**	SQLite, per-step checked + pasted results, debounced autosave.
Report generation	**Very good, now multi-template (Phase 4)**	Evidence built **programmatically**, not by the LLM — the model was observed mis-transcribing a port number. Exam/format templates: OSCP (per-host walkthrough + a proof.txt table spliced from state), CPTS (exec-summary + findings register), H1/Bugcrowd (impact-first + a CVSS 3.1 score *computed*, not asserted). proof.txt/local.txt are real per-host state (auto-captured when a command reads a flag file, or pasted). Collision-proof fences; lab vs REAL-TARGET labelled; detection footprint appended grounded-only; optional red-team OPSEC roll-up (D10).
Assistant chat	**Works**	Session-aware, KB-grounded, deterministic citation extraction (scans the reply for ids that resolve).
Scripts arsenal	**Works**	1,028 scripts deduped from the KB across 6 groups.

Cockpit (execution)

Component	State	Notes
Gated executor	**Excellent design, now properly tooled**	Four ordered gates, mode split, argv-only (never a shell), SSE streaming, run records persisted. The image it execs into now ships ~63 of the 73 catalogued tools (Phase 1).
Isolation gate	**Real**	`assert_isolation_proven()` structurally inspects every attached network for `internal: true` . Not a comment — an actual check.
Engagement mode	**Correct**	Explicit entry required, fail-closed on unknown/exited id, per-command approval re-checked even on the prevalidated path (belt-and-suspenders in `iter_run`).
Scope model (`scope.py`)	**Good**	Hosts, `*.wildcards` , CIDRs, `!exclusions` , `*` opt-out. Fail-closed. No DNS at match time (avoids the "resolve an out-of-scope name to decide it's out of scope" leak). The `_looks_like_host()` false-positives are fixed (Phase 3).
Recon-driven expansion	**Good**	Mines run output for hosts, sorts by scope, in-scope join the live allowed set, out-of-scope surfaced read-only. Cannot widen the scope. Capped and never silently truncating.
Orchestrator loop	**Now state-grounded**	Proposes one command, never executes. Feeds on the structured state model + live task tree (Phase 2) instead of stdout tails.

Component	State	Notes
Engagement state model	**New (Phase 2)**	hosts/services/endpoints/credentials/findings, upsert-only, fed by output parsers + loot-file ingest; drives the planner and a UI panel. Executes nothing (AST-asserted).
`:kali`	**Now a persistent shell (Phase 3)**	One long-lived `docker exec -i sh`; `cd` /env/background jobs persist. Same containment (hardcoded open container, human-only, audited, no isolation gate). Deliberately no PTY — that is what keeps its transcripts clean (§1.1).
`:terminal`	**Real PTY, second surface (Phase 5)**	`pty.fork()` inside the same open box, driven over a framed WebSocket to xterm.js; full-screen tools render and resize. Containment mirrors `:kali` point for point; the raw stream is audited. Does not replace `:kali` (§1.1).
`:exploits`	**New (Phase 5)**	Version-keyed CVE → exploit lookup over the sandbox's local exploit-db catalogue — 47k exploits, 25k distinct CVEs. Read-only; executes nothing.
Live sessions	**Well-designed, now tooled**	Start is a gated command; stdin is human-only and source-scan locked.
HTTP repeater	**New (Phase 4)**	Compose/send/replay/diff. Argv-only curl inside the hardcoded open box (no shell parses a request field; body on stdin), human-only + source-scan locked like `:kali`, scope-checked against a named engagement, every send run-recorded.
Pivot / tunnels	**New (Phase 4)**	chisel/ligolo-ng lifecycle (human-only start/stop) + pure route resolution and a **visible** proxychains rewrite applied *before* the approval screen. A tunnel's subnet enters scope only via an explicit, audited amendment; recon expansion still cannot widen.

AD graph + orchestration

Component	State	Notes
BloodHound parser	**Genuinely good**	Handles v4/v5/CE, zip/dir/json/bytes/mapping, reconciles naming drift, synthesizes DCSync from `GetChanges` + `GetChangesAll`, emits coverage warnings for missing collection methods. Real work.
Path engine	**Correct**	BFS shortest path over abusable edges only, abuse-rank tie-break, k-shortest-ish alternatives.
Technique catalog	**Good**	25 edge kinds, KB-grounded with catalog fallback, target-type specialization (`GenericAll` on a group → `AddMember`). Each edge now also carries a **native Windows variant** (PowerView/Rubeus/Mimikatz) for live WinRM execution alongside the Linux impacket/evil-winrm one.
Orchestrator	**Excellent safety design**	The model picks an **edge index**, never a command. Cannot invent a host, cannot author a command, cannot reach an edge outside the collection. A pick outside the list is refused rather than repaired. Now runs **live** — the approved command executes over WinRM on a selected Windows target (proposes, never auto-fires; regression-locked for the WinRM path too).
`advance` endpoint	**Excellent**	Advancement requires a `run_id` that was **approved** and **exited 0**, verified server-side — transport-agnostic, so the WinRM path advances the walk identically.
Execution tooling	**Now present + executes live**	impacket, certipy-ad, bloodyad, netexec, evil-winrm, bloodhound.py, kerbrute, responder, mitm6 are in the image (Phase 1). The **WinRM driver** (Windows execution backend) now runs the abuse **live** against a real Windows/AD box you run in VMware — the graph is no longer synthetic-only. Live verification against a real box is deferred until that VM is up (build + unit tests are hermetic, mocking WinRM).

Detection footprint

Read-only, honest, well-sourced. Real SigmaHQ rule UUIDs, real ATT&CK ids (Enterprise v19.1), and a verifier script (`pipeline/detection_sources.py --verify`) that re-checks every id against live upstream. The "describes what blue sees, never how to be seen less" line is enforced in code. **Scale:** ~133 command specs, 19 distinct ATT&CK techniques — a good purple-team *flavour*, not a coverage tool.

CRTP conflict — now reconciled (Phase 4, D10). CRTP includes AMSI/logging evasion, which the panel used to refuse outright. It now has an **additive second channel**: the blue-team detection view is byte-for-byte unchanged and still passes the never-prescribe guard, and a separate, opt-in `opsec` channel adds the offensive half — what makes a command loud, the quieter tradecraft, and, mandatorily, *what still records it*. The OPSEC channel has its own guard (it may never advise disabling/clearing/tampering with a sensor); the LLM may extend it for uncatalogued commands, marked `ai_suggested` . See Part III, Phase 4.

Code scan (SAST)

Well-built and genuinely safe. Static-only invariant asserted at every `_spawn()` and again by `test_codescan_safety.py` . Deliberately orthogonal — imports nothing from the engagement/executor/scope model. **Limits — now widened (post-assessment):** the offline bundle grew from 19 rules (Python/JS/TS) to **34 across 8 languages** — Java/Go/PHP/Ruby/C# added (`rules/hackpit-languages.yaml` : command injection, SQLi, unsafe deserialization, code-eval, file-inclusion, SSRF), plus a **ruleset picker** (bundled / per-language / a registry pack for the full online catalogue). Still offline-first. See "Post-assessment refinements".

Frontend

16 routes, real API wiring throughout, SSE streaming, no mocked data layer (`cockpitSample.ts` is the only sample content, explicitly labelled). New in Phases 1–3: the arsenal availability band, per-run time-budget + detach controls, the engagement-state/task-tree panel, the credential-vault "use" action, and the persistent `:kali` shell UI. **Gaps:** no global current-target/engagement context, no engagement export/import, no multi-target view, and the accepted 10-error `react-hooks` lint baseline (documented; `next build` passes exit 0).

---

3. Knowledge base assessment

2,617 entries (1,601 at the time of the assessment; +66 in Phase 4, +950 in Phase 5) · **median body 3,000 chars.**

Source distribution:

Source	Entries
hacktricks	715 (45%)
madstuff	260
some-hacking-resources	233
writeups	59
payloadsallthethings	49
claude-bug-bounty	46

Source	Entries
peh-notes	43
htb-writeups	41
claude-red	39
oscp-cpts-notes	36
decepticon	33
htb-academy	13
hackpit-authored	**12**
htb-box-pdfs	9
galaxy-checklist	7
htb-my-resources	5
shodan-dorks	1

Phase 5 added: hacktricks-cloud **578** · portswigger **372**.

Tiers: 2,265 tier-3 · 241 tier-2 · **111 tier-1** (your own) — see finding 2. **Ranking rebalanced (post-assessment):** tier is now a *substance-gated* nudge, not a blanket trust prior — a thin tier-1 stub no longer outranks richer, more relevant lower-tier content, and a command-rich page wins close calls regardless of tier. See "Post-assessment refinements".

Findings

1. **PayloadsAllTheThings payload depth — now recovered (Phase 4, D11/D12).** PATT's 66 `.txt` payload lists + the shodan dork list are ingested at **payload-level granularity** (64 `payload-set` + 2 `dork-list` entries, `no_merge`-guarded so consolidation can never collapse them again), each a searchable entry over a full sidecar corpus mounted read-only at `/payloads`; 56 `oscp_tools` files went to the Scripts Arsenal. KB 1,601 → 1,667. See Part III, Phase 4.
2. **"Grounded in your own notes" is mostly "grounded in HackTricks."** Search boosts tier-1 (`search.py:58`), but with 111 tier-1 entries that lever has little to pull on. **Growing tier-1 is the highest-leverage KB work still available** — **still open**, and now the *only* open KB item. Phase 5 grew the KB by 950 entries, all tier-3, so the ratio moved the wrong way: the fix is writing, not ingesting.
3. **Empty categories — mostly fixed (Phase 5).** Was: cloud 2 · mobile 2 · iot 2 · forensics 1 · ics 1 · phishing 1 · supply-chain 1. Now **cloud 535** and **supply-chain 47** (HackTricks Cloud), and web/API coverage is deep (PortSwigger). **Still thin:** mobile · iot · forensics · ics · phishing — none of which is on the target list, so they stay unfilled by choice rather than oversight.
4. **HTB Academy — narrowed, not closed (Phase 5).** The local folder is an 18-module *slice*, not the CPTS syllabus, and HTB content is proprietary so nothing is scraped. Auditing the slice against what had been ingested found two files silently lost — one tracked in git but absent from disk, one lost purely for having a `.txt` extension — both now recovered. Closing this properly needs the course itself.
5. **Missing as retrievable topics — fixed (Phase 5).** API security (GraphQL/REST), OAuth/SSO, race conditions, business logic, request smuggling, prototype pollution, deserialization and SSRF→cloud-metadata chains all

now resolve, most to PortSwigger Academy material with the lab that drills each one alongside.

6. **No CVE/exploit index — fixed (Phase 5).** `:exploits` answers service+version → CVE → public exploit over the sandbox's own exploit-db catalogue, version-compared rather than substring-matched.

Item 2 is the only KB item still open. Items 3–6 were the recommended next batches; they were never in scope for the first four phases and were built in Phase 5.

Recommended ingestion, in priority order — status

1. **PortSwigger Web Security Academy** — the single best web-security source, and structured. **Done (Phase 5, +372).**
2. **HTB Academy modules** properly (CPTS syllabus). **Partially — the local slice is fully ingested; the syllabus needs the course.**
3. **HackTricks Cloud. Done (Phase 5, +578).**
4. **PayloadsAllTheThings re-ingested at payload-level granularity** (see §4). **Done (Phase 4).**
5. **An exploit-db / CVE mapping** keyed on service+version. **Done (Phase 5) — and deliberately not as KB prose; see Part III.**

---

4. Source-usage audit — "did I use my sources properly?"

Measured on disk against the built KB, not inferred.

4.1 The headline finding: the pipeline only reads `*.md`

`pipeline/ingest.py:229` → `sorted(source_path.rglob("*.md"))`; `pipeline/ingest_notes.py:599` likewise. **Every `.txt`, `.json`, `.py`, `.sh`, `.csv`, `.yaml` file in every source tree was invisible to the pipeline.**

Source folder	Files	`.md`	KB entries	Non-md skipped	Of which is *real content*
`hackdic`	807	806	715	1	—
`madstuff`	798	797	260	1	—
`some hacking resources`	233	233	233	0	—
`PayloadsAllTheThings`	481	134	49	347	**66 `.txt` payload lists + 28 `.py` scripts**
`oscp-cpts-notes`	288	85	36	203	174 PNG screenshots (+ git internals)
`oscp_tools`	99	1	0	98	28 `.exe` binaries, ~22 `.ps1` / `.sh` / `.py` scripts
`HTB_academy`	48	16	13	32	3 PNGs — effectively nothing
`shodan-dorks`	32	1	1	31	**1 `.txt` dork list**
`Hacking-Tools`	31	1	0	30	nothing — git internals only

Source folder	Files	`.md` KB entries	Non-md skipped	Of which is *real content*
`PRACTICAL ETHICAL HACKING NOTES`	110	45	43	65	images
`more new resources` (writeups)	118	72	100	46	images
`htb my resources`	5	5	5	0	—
`htb boxes`	10 (pdf)	0	9	—	PDF path handled separately
`PentestTools`	**0**	0	0	—	**empty folder**

Scale: stripping git internals, images and compiled binaries, the genuinely lost knowledge was roughly **95 files** — chiefly **PATT's 66 `.txt` payload lists** (the pipeline ingested the *explanations* and discarded the *payloads* — backwards for bug bounty), the **shodan-dorks `.txt`**, and **~22 `oscp_tools` scripts**. **Fixed (Phase 4, D11/D12):** an additive Route-B ingester (`pipeline/ingest_corpora.py`) folded all 66 payload lists + both dork lists into the KB as `payload-set` / `dork-list` entries with full sidecar corpora, and 56 `oscp_tools` files into the Scripts Arsenal — without rewriting a single existing entry (verified byte-identical) and idempotently. Two env traps hit and handled: `rglob` silently omitted 2 of 66 files (OneDrive dehydration → union with `git ls-files`) and 22 files were AV-locked (→ git-blob recovery).

4.2 Consolidation is lossy for some sources

`madstuff` 797 md → 260 (33%). `oscp-cpts-notes` 85 md → 36 (42%). `PayloadsAllTheThings` 134 md → 49 (37%). Some is correct deduplication into HackTricks; a 3:1 collapse on your OSCP/CPTS notes is worth an eyeball — those are exactly the entries you'd want surviving as tier-1/tier-2.

> **Audited (post-assessment) — healthy dedup, no loss.** A read-only provenance audit of the OSCP set: **36 survived as their own entries (33 with real commands)**, **12 folded into stronger canonical pages** (PayloadsAllTheThings/peh-notes/some-hacking-resources — preserved as `also_covered_in` + variants, not deleted), and the remaining ~37 were **same-technique OS/lab splits merged together** (the GitBook ships Linux/Windows/lab variants as separate files). **Zero** OSCP entries were dropped as low-value (only 11 were dropped KB-wide). Also corrected: `oscp-cpts-notes` is a **tier-3 public GitBook repo**, not your own writing — your genuine tier-1 comes from `writeups` / `peh-notes` / `htb-my-resources` . **Decision: no re-ingest** — re-adding with `no_merge` would just recreate duplicate technique pages that hurt search. See "Post-assessment refinements".

4.3 The biggest missed idea — PentestGPT's Pentest Task Tree — is now BUILT

The assessment's single highest-value reasoning-layer recommendation was PentestGPT's task tree: a *persistent, hierarchical, status-tracked engagement state the model reads from and writes back to every turn.* **This shipped in Phase 2** (`backend/state/tasks.py`) — layered `1 / 1.1 / 1.1.1` tasks with `todo | done | n/a` , updated as results arrive via model-proposed operations that code validates. It is retained here as the rationale for what was built. (Implementation note: HackPit's variant has the model return *validated operations* rather than rewriting the whole tree, so one bad response cannot silently rewrite the plan.)

4.4 The structural pattern

The pipeline is excellent at *techniques* and lossy on *lists, workflows and payload corpora*. A checklist is a *sequence*, a dork list is a *set*, a payload collection is a *corpus* — all three were forced through a technique-shaped, markdown-only schema and collapsed. **Both fixes shipped (Phase 4):** the Route-B ingester accepts non-markdown inputs, and the second entry shapes (`meta.kind` = `payload-set` / `dork-list` , `no_merge` -guarded) survive consolidation intact. The `checklist` shape is defined and available; no checklist source was in the D12 scope, so none were ingested yet.

---

5. What is genuinely excellent — do not refactor

- **The safety architecture is real.** Source-scan regression locks on `:kali` and session stdin, four ordered gates, clean lab/engagement mode split, `assert_isolation_proven()` as a structural runtime check. Better than most commercial tooling. (The Phase 1–3 work preserved every one of these invariants; the test suite grew with it.)
- **The grounding discipline is the best idea in the project.** Grounded-vs- `ai_suggested` labelling; entry_ids must resolve or the step is downgraded; commands come from the KB, never the model; **evidence spliced programmatically into reports rather than written by the LLM**; `foreign_refs` as an honesty marker; the Channel-2 leak guard.
- **The AD orchestrator picks an EDGE, not a command.** A safety design that also improves output quality. The Phase-2 task tree follows the same principle (validated ops, not free-form rewrites).
- `advance` **requires an approved, exit-0 run** — evidence-based state transition, verified server-side.
- **The consolidation pipeline** — alias-key + cosine match, structural merge, idempotent, provenance preserved.
- **The detection footprint's framing and sourcing** — real Sigma UUIDs with a drift verifier.
- `substitute_target` — CIDR sentinel, host-position gating, refusal to rewrite what it can't rewrite confidently.
- **The BloodHound parser** — v4/v5/CE reconciliation with honest coverage warnings.

---

6. Security posture of HackPit itself

- **No authentication anywhere.** No `Depends` , no key, no session. CORS restricted to `localhost:3000` . Honestly documented in code.
- On a laptop this is fine. **On a VPS attack box it is an unauthenticated RCE endpoint** — `POST /cockpit/kali` (and the new persistent-shell routes) run arbitrary `sh -c` in a container with full network reach to your host and LAN. Auth is a hard blocker before any non-localhost deployment.
- Secrets handling is correct: `llm_config.json` gitignored, API key written to disk but never returned, collector preview redacts the password/hash. The engagement-state DB (which now holds **real captured credentials**, by decision) is gitignored like the rest.
- The repo is code-only; KB, sources, sessions DB and `.env` are all gitignored. A prior full-history audit confirmed no secrets were ever committed.
- **Known:** a self-scan flagged SSRF and CORS misconfiguration in HackPit's own backend (memory obs #1496). Worth revisiting before any exposure.

---

7. What the assessment examined

Read in full: every non-test backend module (executor, allowlist, config, sandbox, engagement, scope, kali, session, runstore, router, models; attack_path, main, orchestrator, llm, report, chat, context_channel, sessions; all of adgraph, arsenal, detection, codescan), `pipeline/search.py`, `docker/Dockerfile.sandbox`, `docker-compose.yml`, the QA report, and the live KB.

Structurally mapped: the frontend (routes, API surface, component→endpoint wiring, demo/mock scan — none found beyond the labelled `cockpitSample.ts`); the test files; `pipeline/consolidate.py`.

Source trees inspected on disk: `hexstrike-ai` (151 `@mcp.tool` defs), `mantishack`, `Decepticon`, `PentestGPT` (task-tree prompt read directly), `claude-red`, `claude-bug-bounty`, `Galaxy-Bugbounty-Checklist`, `hackdic`, `htb boxes`, the writeup trees, `some hacking resources`. File counts and markdown ratios measured per tree.

Verified live (assessment): container tooling (37-tool probe), SYN-scan capability, nuclei templates, container capabilities, KB statistics, ingester file-type globs.

PART II — WHAT REMAINS

Decided with Zaid, 2026-07-26. All five phases and every decision that drove them are in Part III. The build is complete; this section is only what is intentionally left — some of it deferred, some of it rejected outright.

Standing policy (unchanged, still governs the project)

These are not open work — they are the rules the project runs by, and building is finished, so they no longer need a decision table. Kept here as the active policy:

- **D2 — Windows + Docker Desktop; a VPS is added later, only for bug-bounty callbacks.** Docker Desktop is a real Linux VM (WSL2); the one thing it can't give is a publicly reachable listener for blind SSRF/XXE/RCE. The repeater (Phase 4) sends and reads the *direct* response; the VPS piece is still deferred until bounty work needs it.
- **D5 — Per-command approval stays. Risk-tiering is rejected, not deferred.** One-keystroke approve shipped (Phase 3). Tiering — auto-running "passive" commands, batch-approving read-only plans — was reconsidered and **decided against** (§1.2): in engagement mode per-command approval is the *only* thing bounding where a command may go, and a classifier deciding when to skip the human is a new component on the wrong side of that boundary. Every execution surface since preserves this — repeater, tunnels (rewrite *visible before* approval), and Phase 5's `:terminal` (human-only, no agent path).
- **D8 — HexStrike is a reference, never an execution backend.** Its tools bypass all four gates, the target-lock, the scope model and the audit trail. Rejected, not deferred.
- **D9 — CRTP Windows execution: CLOSED via a WinRM driver against an external VM (was "accept the gap").** The original decision accepted that CRTP's PowerShell/.NET tooling can't run on the Linux container. That gap is now closed the honest way: **no Windows VM lives inside the project** — HackPit *drives* one you run in VMware over WinRM (Model A), a new execution transport behind the same gates. Windows-only tools now report runnable when a Windows target is selected (reconciled per active target); on a Linux run they are still N/A and marked. The AD graph executes live against that target. See the Windows execution backend in Part III.

Explicitly deferred (with the reason)

- **A VPS for blind out-of-band callbacks** (D2) — a NAT'd laptop has no public listener; deferred until bounty work needs it.
- **A Windows execution target for CRTP** (D9) — **DEMONSTRATED LIVE** (build #9, 2026-07-31): the WinRM driver executes CRTP/AD work live on an external VMware VM you run. No longer "built but only unit-tested" — a real PowerShell round-trip lands on the profile host over WinRM, the whole-script danger classifier fires on real input, the credential leaks into nothing, and output ingests into state (Task 1, 14/14). See build #9 in Part III. The one caveat still open is a full Windows-target C2 callback (Task 4), which needs a routable listener.
- **The AD graph run off real collection, not synthetic data** — **DEMONSTRATED LIVE** (build #9): the gated, scope-locked `bloodhound-python` collected the real `corp.local` (84 nodes / 625 edges), it parsed into the typed graph rather than `sample_data`, and the route to Domain Admin was computed on the real graph (Task 2, 10/10). The live abuse walk then executed a real DCSync over the domain — four NTLM hashes including `krbtgt` — through the danger red-confirm, with real loot ingesting into state and the walk advancing only on

the approved exit-0 run (Task 3). Four defects that only a real domain could surface were found and fixed (credential-in-record leak, impacket parser-name mismatch, KB grounder arming a free edge, collector FQDN classifier), each regression-locked. The native-tooling abuse variants (Rubeus/mimikatz *on* the DC) stay not-run: a bare Server 2022 has no offensive tooling and staging it was out of scope.

- **Growing tier-1** (§3, finding 2) — the only KB item still open, and the one that cannot be solved by ingesting: it needs Zaid's own writing.
- **The HTB Academy syllabus proper** (§3, finding 4) — the local slice is fully ingested; the rest is proprietary and needs the course.
- **Thin categories with no target on the list** — mobile · iot · forensics · ics · phishing (§3, finding 3). Unfilled by choice.
- **C2-framework install + the persistence evasion/OPSEC half** — **DONE** (build #4, no longer deferred): Sliver is installed and wired to a gated panel, and the persistence OPSEC notes were backfilled. Empire and weevely remain catalogued reference-only by choice.
- **Live-fire verification of the C2 and tunnel surfaces (build #4) — DEMONSTRATED IN LAB, with exact caveats.** A controlled lab exists (`docker/proof/live_fire_proof.sh` + `live-fire-lab.yml` , four containers on `internal: true` networks) and drives the surfaces through their shipped gated entry points: **66 passed, 0 failed, 3 not-run.** What is demonstrated live: an implant that actually builds and **a beacon that calls back**; a **proxychained request routed through the pivot** into a subnet the operator box has no route to, using the argv `wrap_command` produced; all three lifecycle gates (Sliver server, both pivot kinds, DNS listener) refusing on all three limbs with nothing spawned; every listener coming up, being confirmed bound by `ss` inside the container, and staying bound for the whole phase; the pivot subnet entering scope only via the explicit amendment, with the deep target confirmed unreachable beforehand; the DNS key absent from the pasteable one-liner; and **containment holding live, measured from inside the implant host** — no internet, no host, C2 only. What is **not** demonstrated, and why, kept deliberately separate from the above:
  - **The three defects the first live-fire run found are FIXED (2026-07-28), and the fixes are what the numbers above measure.** The Sliver implant build now runs through the console `sliver-server` hosts (Sliver 1.5 has no `generate` subcommand on either binary) and decides `generated` vs `failed` by reading the artifact back rather than trusting a console exit code; both listener lifecycles now hold a console binary's stdin open and report a status they **observed**. Fixing them surfaced two more of the same family, also fixed: a stop that killed the `docker exec` client while leaving the server running inside the container (next start: `EADDRINUSE`), and a `proxychains` config shipped pointing at Tor rather than the SOCKS5 port HackPit's own rewrite targets.
  - **iodine's IP-over-DNS tunnel is now demonstrated** carrying traffic through the gated surface, confirmed DNS-encapsulated on the wire (2026-07-28). **dnscat2's handshake is decisively an upstream defect** in the pinned build — `tcpdump` shows the server itself answering `NXDOMAIN` to correct queries, reproduced over loopback in one container — sitting above HackPit's proven-bound listener. What stays operator-side: a **delegated DNS zone** (a domain with an NS record pointed at a publicly reachable listener, the D2 VPS gap), which neither tunnel can exercise without it.
  - **ligolo's interface routing** needs `session` then `start` typed into the proxy's console. HackPit holds that console's stdin open and deliberately never types into it, so completing a ligolo session stays the operator's step; the routed-traffic claim is carried by chisel, whose server needs no console. **Detonating an artifact on an instrumented Windows host** remains untouched (below), and pairs with the D9 VM. The original list, for the record:
  - **Catch a live Sliver beacon.** The server lifecycle starts and stops and is audited, but no implant has ever called back. Needs a listener reachable from a victim host — i.e. the same VPS gap as D2, or a lab VM on a

routable segment. Until then the implant argv is only as correct as the catalog's documented syntax; a wrong flag surfaces as `status='failed'`, not as a containment failure.

  - **Stand up a real DNS tunnel.** `dnscat2-server` and `iodined` start inside the engage sandbox and the client one-liner is handed back, but no delegated zone exists to test against. Needs an authoritative zone the operator controls with an NS record pointed at the listener. `iodined` also needs a TUN device in the engage sandbox — plausible given its `NET_ADMIN`, but unproven.
  - **Run a generated artifact on a Windows box and watch the telemetry.** The evasion engine emits artifacts and the paired footprints claim specific Event IDs and Sysmon codes. Those claims are transcribed from ATT&CK v19.1 and SigmaHQ, not observed here. The honest version of this feature is only fully honest once someone detonates an artifact on an instrumented host and confirms the footprint matches what actually landed in the log. This is the natural pairing with the D9 Windows VM. None of this blocked the build shipping — the containment properties are what the test suite locks, and those are provable without live infrastructure. Build #7 converted the containment half from "locked by tests" to "holds live", and the efficacy half from "documented" to "three specific defects, named and reproducible" and then to "fixed, with the callback and the routed traffic actually observed". What remains before build #4 could be called field-ready on these surfaces is the DNS session and a delegated zone.

- **An independent review of build #4's Tasks 8–13** — **DONE** (no longer outstanding). Run in a fresh session (the safety classifier had begun refusing subagent dispatch partway through the build, and it refuses on accumulated context). It found **no containment hole** — no agent path, no shell, the container never request-influenced, no delivery primitive — and five substantive defects, all fixed: a multi-technique request that built one artifact and described a different one; the T1690/T1685 mis-mapping above; two stub headers describing memory patching when the stubs are managed-reflection bypasses; a no-agent-path test whose positive control counted the wrong variable (99 files reported, 5 actually scanned); and a `_gate_request` comment asserting a scope-gate behaviour that does not hold. Plus seven minor items — an image smoke test that could not fail, unpinned installs, dead fields, weak assertions. Every fix carries a test that fails without it.
- **Gate-integrity Criticals 2 and 3 — DONE (build #5, 2026-07-27).** Both closed; full detail in Part III. **Critical 2** (the WinRM first-token classification — the sharpest known hole in the tool, because it defeated the red-confirm by moving a cmdlet one token to the right on the one path that reaches a real domain-joined host) is fixed by classifying the whole joined PowerShell script, deliberately without parsing it, with the gate and the transport now deriving that string from one shared function. **Critical 3** (source-scan locks covering a fraction of the tree) is fixed by one shared scanner: whole-tree `rglob`, repo-relative allow-lists, an AST pass for indirection, and per-lock planted-violation controls — coverage per lock goes 30 of 69 modules to 67. The plan's third task, the one that matters longest, is written down in `backend/AGENTS.md`. **`I2`, the ungated tunnel listener start, was outside build #5's plan but has since been closed** — `start_tunnel` now runs the real `validate_request` before spawning, `TunnelStartRequest` carries `approved` / `dangerous_ack` defaulting false, and the route 403s a safety refusal; detail in Part III.
- **Pinning the build-#4 install layer** (audit M2) — **DONE** (build #7, no longer deferred). All three float-installs are pinned to real resolved values: dnscat2 to commit `42f8d783…`, the gems to `trollop:2.9.10 salsa20:0.1.3 sha3:2.2.4 ecdsa:1.2.0`, `donut-shellcode==1.1`. `test_evasion.py` now asserts full pinning (three installs, three `ARG` pins, a real 40-char SHA, no floating form left behind) rather than tolerating a declared gap. The fresh build also surfaced a smoke test that **had never passed** — `sliver-server version | grep -qi sliver`, against a binary that prints only `v1.5.42 - <sha>` — so that layer had not built since the check was added; both sliver checks now match `v${SLIVER_VERSION}`, which is strictly stronger. `hackpit-kali:build7` builds clean with every pinned tool resolving and smoke-testing by its real invoked name.
- **The prevalidated path's danger re-check** — **DONE** (build #7). `iter_run(prevalidated=True)` re-checked approval but not danger; nothing was exposed (the single such caller validates first) but the asymmetry was a

trap for the second caller. It now re-checks danger in all three modes using the same per-mode classifier `validate_request` uses, regression-locked in `test_prevalidated_gates.py`.

- **Kali VM as an execution target; HexStrike as an execution backend; risk-tiered / batch approval — rejected, not deferred** (D8, D5, §1.2).
- **Authentication before any non-localhost deployment** (§6) — no decision needed until a VPS enters the picture, but a hard blocker the moment it does. `:terminal` makes this sharper: an unauthenticated *interactive terminal* onto a box that reaches the host and LAN. **Still outstanding, and now quantified (2026-07-31).** A Cloudflare Tunnel + Access + app-level-auth build was scoped in full and then deliberately not taken. The scoping established the exact state of the gap: `backend/main.py` has **no authentication of any kind on any route** — no `Depends`, no HTTPBasic, no API key — and the CORS allowlist is a *browser* policy that stops nothing which is not a browser. The sharpest edge is the single WebSocket route `/cockpit/terminal/ws`, a live PTY into the Kali container: gating HTTP alone would leave a free shell. Localhost-only remains the default and the mitigation.
- Screenshot capture, recon diffing/monitoring, checklist-driven runs.

reconFTW — recon gaps mined, and what to skip

reconFTW (six2dez, MIT) was reviewed for what HackPit's *recon* was missing — its thinnest area (the incorporation narrative is in Part III). Respecting the constraint that governs everything here — HackPit is human-gated, so "incorporate" means adopt reconFTW's **knowledge**, never its auto-runner — the concrete gaps, where each landed, and rough effort:

- **URL-triage pipeline** — `gf` + `qsreplace` + `unfurl` (arsenal `web`) + a KB methodology entry. The single biggest bug-bounty gap: HackPit harvested URLs but had no triage step. *Low — done.*
- **Subdomain permutations** — `gotator`, `regulator`, `subwiz` (arsenal `recon`). An entire enum technique HackPit had none of. *Low–med — done.*
- **Mass resolution** — `puredns` + `massdns` + `dnsvalidator`, with a baked resolver list (arsenal `recon` + image). HackPit had only `dnsx`. *Med — done.*
- **JS mining** — `jsluice`, `subjs`, `getjs` (arsenal `web`): endpoint/secret extraction, not just crawling. *Low–med — done.*
- **ASN → CIDR expansion** — `asnmap`, `mapcidr` (arsenal `recon`) for wide-target scope work. *Low — done.*
- **External OSINT** — `trufflehog`, `github` / `gitlab-subdomains`, `dorks_hunter`, `gitdorks_go`, `msftrecon`, in a **new `osint` category** (HackPit had no OSINT home). *Low — done.*
- **Bucket/cloud enum** — `s3scanner`, `cloud_enum` (arsenal `cloud`): external asset discovery, distinct from the posture scanners. *Low–med — done.*
- **Injection completers** — `commix`, `SSTImap`, `nomore403`, `crlfuzz` (arsenal `web`): fill the cmd-injection / SSTI / 403-bypass / CRLF tool gaps (skills existed; tools didn't). *Low — done.*
- **Recon ordering** — the subdomain-enum sequence and the URL→gf-triage→fuzz pipeline, as two KB `checklist` entries (`recon-methodology-*`) + an advisory note in the composer's real-target prompt. The higher-leverage half — tools without the ordering is the cheap half. *Low–med — done.*

Skip — conflicts with the human-gated model (rejected, not deferred): the `reconftw.sh` runner and `reconftw.cfg` engine (autonomous chaining is the exact thing HackPit rejects), `interlace` (parallel command execution), `notify` (autonomous alerts), monitor/incremental/diff auto-rescan, the axiom/Ax fleet, `reconftw_ai`

and faraday. `inscope` is redundant — `scope.py` is stronger (fail-closed, no DNS at match time). `interactsh` /OOB stays deferred on the VPS (D2, unchanged). `waymore` was considered and left out: `gau` + `waybackurls` + `katana` already cover URL harvesting.

PART III — BUILD LOG (Phases 1–5, shipped 2026-07-26)

Branch `sandbox-kali-image`. Full hermetic safety suite green throughout (36 test files); both Docker proofs 4/4 (lab still egress-less; engage fully open); browser-verified with Ollama (`qwen3:8b`).

Decisions taken (D1–D20)

Every decision that drove the build. D1/D3/D4/D6/D7 were the Phase-1 build; D9 was a standing accepted gap that is **now built** (the Windows execution backend); D2/D5/D8 are standing policy (Part II); D10/D11/D12 were Phase-4 work, now built; D13 is this document; D14/D15 shaped Phase 5; **D16/D17 are build #4** — D16 amends D10 and is the one policy reversal in the project's history.

- **D1 — Fix the sandbox image; HackPit *is* the attack box.** A Kali VM's advantages (root, raw sockets, VPN, `/etc/hosts`) are all reachable in Docker via capabilities + `/dev/net/tun` . *Built (Phase 1).*
- **D3 — Capabilities + root on the ENGAGE sandbox only.** Lab keeps `cap_drop: ALL` + non-root. *Built.*
- **D4 — All three sandboxes get the new toolset; only the lab's *network* isolation stays.** *Built.*
- **D6 — Catalog describes reality, not aspiration.** *Built.*
- **D7 — Startup reconciliation check.** *Built.*
- **D9 — CRTP Windows execution.** Was "accept the gap; no Windows VM." Now **closed by the WinRM driver** (Windows execution backend): HackPit drives an external VMware VM you run — no Windows VM inside the project. Windows-only tools run when a Windows target is selected; still N/A + marked on a Linux run. *Built (the AD graph executes live).*
- **D10 — Allow evasion/OPSEC content as an additive second channel, keeping the blue view.** *Built (Phase 4).*
- **D11 — Fix the KB gap via Route B (additive merge), not a rebuild.** *Built (Phase 4).*
- **D12 — Ingest PATT payloads + shodan dorks into the KB; `oscp_tools` into the Scripts Arsenal.** *Built (Phase 4).*
- **D13 — Plan lives in this document.** *Done.*
- **D14 — The PTY is a SECOND surface, never a replacement.** The old framing treated auditable transcripts and full-screen tooling as alternatives. They are only alternatives on one surface. `:kali` keeps its sentinel; `:terminal` gets the pty; both are audited and identically contained, and a test asserts `kali.py` never grows a pty. *Built (Phase 5).*
- **D15 — The CVE index is a lookup table, not a KB batch.** "Find the exploit for this version" wants an exact, deterministic table; embedding it as prose would make the one query it exists to answer fuzzy. Its own data shape, search path and surface — and it executes nothing. *Built (Phase 5).*
- **D16 — Lift the OPSEC sensor-tamper ban; the honesty marker becomes the sole invariant.** Amends D10. The OPSEC channel previously refused any sensor-blinding phrasing, which became incoherent once build #4 shipped an engine that emits AMSI-patch and ETW-blind artifacts: the tool would have been producing artifacts it was forbidden to describe. The ban is lifted **entirely**, and two invariants carry the whole weight

instead — every note must name what **still records** the activity, and the blue-view footprint is always produced alongside and can never be suppressed. The blue-side describe-never-prescribe guard and all four execution gates are **unchanged**. A posture shift on a public repo, taken deliberately. *Built (build #4).*

- **D17 — Split the gating for C2 rather than gate it uniformly.** Server and listener **lifecycle** is human-only with no red-confirm (operator infrastructure on the operator's own sandbox — no target is touched, so clicking start is the approval); **artifact generation** is a fully gated command (approval + scope + red-confirm) because it produces something that will run on someone else's machine. Uniform gating would have meant either a meaningless confirm on every server start or no confirm on a payload build. *Built (build #4).*
- **D18 — The loop may REASON, but it still never executes, and approval stays per-command.** Build #8 makes the proposer genuinely deeper — working memory (a tried/failed ledger), hypothesis-first proposals, a scored candidate frontier, failure diagnosis, a refute-first critic, domain-specialist routing, fingerprint-keyed retrieval, and a model-tier config lever. Every one of those produces **proposals and rationale only**. The decision recorded here is the boundary they were built against and re-locked to: `orchestrator.py` and the whole new `reasoning/` package have **no path to any execution surface**, the frontier holds untried leads rather than a queue that fires, and there is **no auto-approve / batch-approve / run-the-chain** anywhere — a human still approves every single command. Deeper proposer, identical autonomy. *Built (build #8), regression-locked by an extension of the loop's source-scan onto the whole reasoning package.*
- **D19 — The launcher lives ON the home page, not on its own route.** Roughly a dozen surfaces were built and then invisible — `:arsenal`, `:c2`, `:tunnels`, `:windows`, `:repeater`, `:scripts`, `:code-scan` and the AD graph were reachable only by typing the URL, and the operator's own notes had flagged it twice before it was fixed. Both placements were built as mocks and compared. The deciding argument was that *a page you have to navigate to fixes discoverability only if you remember to navigate to it* — so the index belongs on the screen you already land on. Cost measured before committing: +1,024px on a page already ~1,936px, i.e. ~2 screens to ~3. The hero, the stat counters and the intro are untouched. *Built (build #11).*
- **D20 — Operator identity is configuration, never a constant.** This repo is public, and a real name, email or OSID written into source is in the public git history permanently. Identity therefore lives in a gitignored `backend/operator.json` with env overrides, read through two deliberately separate accessors: `public_profile()` (name + handle) for the browser, `report_identity()` (adds OSID / handle / contact) for a report handed to an examiner. A page has no use for an OSID, so it never receives one. The report block is *spliced by code*, never prompted for — handing the model a real name or an OSID invites it to transcribe or invent one. *Built (build #11).*

Phase 1 — reach a real target

Commits `e4e8de5`, `80a18d5`, `2d0928f`.

- **New sandbox image** (`docker/Dockerfile.sandbox`) on `kalilinux/kali-rolling` + `kali-linux-headless`, with SecLists, the Kali `wordlists` set (rockyou ungzipped) and **~13,400 nuclei templates** baked in (the lab has no egress, so nothing can be fetched at runtime). Thematic tool layers for the specific tools the assessment found missing — impacket, netexec, evil-winrm, bloodhound.py, certipy-ad, bloodyad, responder, mitm6, smbmap, enum4linux-ng, gobuster/nikto/feroxbuster/subfinder/httpx/amass, metasploit, exploitdb, hashcat/john, chisel/ligolo-ng/proxychains/socat/ncat, openvpn — plus katana/kerbrute/waybackurls/dalfox/gau/rustscan/jwt_tool as pinned release binaries (no Kali package). **Result: ~63 of 73 catalogued tools present.** Kali's setcap'd `nmap` / `fping` are stripped at build so `no-new-privileges` doesn't refuse to exec them.

- **Privilege split** (`docker-compose.yml`) — the **engage** sandbox runs `user: root` with Docker's default capability set + `NET_ADMIN` and `/dev/net/tun`, so `-sS` / `-sU` / `-O`, masscan, tcpdump, privileged-port binds (responder/ntlmrelayx) and VPN all work. **Lab and `:kali` are untouched** — uid 1000, `cap_drop: ALL`, no devices. Verified: engage does SYN scans and binds :445; lab/ `:kali` refuse `-sS`.
- **Per-request timeouts** (default 180s, ceiling 3600s) on `/cockpit/exec` and `:kali`; **background/detached jobs** returning a run_id and a reconnectable replay stream (`/cockpit/runs/{id}/stream`), with a reaper. `:kali`'s old hardcoded 60s is gone.
- **Loot volume** — `backend/data/engagements` → `/loot` on the engage and `:kali` sandboxes (deliberately **not** the isolated lab); each engagement works in `/loot/<id>` via `docker exec -w`, so `nmap -oA` output survives `docker compose down`.
- **Startup reconciliation** (`GET /tools`) — probes the running sandbox with one `command -v` sweep over every catalogued name+alias, filters the planner's prompt to installed tools, and surfaces the gaps in the arsenal UI. Windows-only tools (rubeus/powerview/mimikatz/winpeas) are marked `platform: windows`, kept for planning/write-ups, and never proposed.
- **Arsenal catalog** reconciled against the shipped image; the orchestrator prompt no longer hardcodes `gobuster` / `nikto`.

Phase 2 — make it remember

Commits `c606f6d` (backend), `a8f40ae` (UI).

- `backend/state/` **package** — the structured engagement state the assessment identified as the deepest gap: `hosts → services`, `endpoints`, `credentials`, `findings` as dataclasses over a shared SQLite store where **every write is an upsert** (re-running a scan corrects, never duplicates). Output **parsers** (nmap XML, httpx/ffuf/nuclei JSON, secretsdump) as pure functions, and a post-run **ingest** that reads both a run's stdout *and* any file it wrote into its loot directory (that is how `nmap -oA` populates state). The package **executes nothing** — AST-asserted, because ingest runs automatically after every command.
- **Pentest Task Tree** (`state/tasks.py`) — PentestGPT's central idea (§4.3): layered `1 / 1.1 / 1.1.1` tasks with `todo | done | n/a`, persistent per session, updated as results arrive. The model returns **validated operations** (`add_subtask` / `mark_done` / `mark_na` / …) that code applies against the stored tree; one bad op is rejected on its own and the rest still apply.
- **Orchestrator grounding** — the proposer prompt now leads with the accumulated state + live task tree, with raw output demoted to a short tail. Measured: **460 chars of state vs 6,600 of stdout tails** for 12 runs of one nmap, and each fact appears once instead of once per run. A tail window forgets; state does not.
- **Credentials in the prompt: real values** (Zaid's explicit decision) so the planner writes fully-formed commands — one constant (`render.INCLUDE_CREDENTIAL_SECRETS`) with a test proving the flip works, so the choice stays reversible.
- **UI** — the `CockpitState` panel: counters, the task tree (with "seed from plan"), hosts+services grouped, endpoints with status colours, credentials masked with per-row reveal + validated/untested, findings by severity.
- **Bonus fix found by the browser pass** — the arsenal `/arsenal` response model was silently dropping `platform` / `runs_here`, so every tool badged "windows only". Fixed + regression-tested.

Phase 3 — make it fast to drive

Commit `b316b74`.

- **Step 11 — one-keystroke approve.** The guided loop takes `Enter` = approve, `S` = skip, `Esc` = stop, hints shown on the buttons. A dangerous proposal never fires on `Enter` (the explicit danger confirm is still required); shortcuts are ignored while a text field is focused.
- **Step 12 — `_looks_like_host()` fixed.** Flipped from "assume host unless the last segment is a known file extension" to a **positive test**: a dotted token is a host only if its last label is a plausible alphabetic TLD and not a file extension. Stops the false rejections (`directory-list-2.3-medium` , `Mozilla/5.0` , `-oA scan.1.2` , version strings) while still catching real hosts; a genuine off-target host is still refused. This was the prerequisite for any future auto-run tier.
- **Step 13 — persistent `:kali` shell.** One long-lived `docker exec -i sh` instead of a fresh exec per command; each command is delimited over the pipe with a per-command sentinel so output and exit code read back cleanly. `cd` /env/background jobs persist (verified in-browser: `cd /tmp` → `export FOO=…` → `echo cwd=$(pwd) foo=$FOO` returned `cwd=/tmp foo=…` across three separate API calls). Every `:kali` containment invariant intact. No PTY, by design (§1.1). (Caught the Windows `\n` → `\r\n` stdin-translation bug in the process — the Linux shell was receiving `pwd\r` ; fixed with bare-LF stdin.)
- **Step 14 — credential vault.** Captured credentials fill the `<user>` / `<password>` / `<ntlm-hash>` / `<domain>` placeholders the AD templates carry, one click instead of retyping a hash. The placeholder→field mapping lives server-side (`state/credvault.py`) so front and back can't drift; a password fills `<password>` not `<hash>` and vice-versa; only credential placeholders are touched. Verified in-browser: "use svc_sql" turned `nmap -u <user> -p <password> …` into `nmap -u svc_sql -p S3cr3t! …` .

Phase 4 — content & goal-specific

Commits `90fe260` , `885d776` , `bf46695` , `8486c56` , `937c420` .

- **Item 1 — KB Route B additive ingest (D11/D12).** `pipeline/ingest_corpora.py` — a targeted ingester over only the missing files, run *additively* on the built KB (never through the real pipeline, which would revert downstream enrichment and rewrite a 15 MB gitignored, Defender-sensitive file). Two regression-locked guarantees: existing entries pass through as **bytes** (no key/escape drift on the 1,601 it never looked at), and every line it owns carries `meta.corpus_ingest` so a re-run is byte-identical. Shapes (§4.4): **64** `payload-set` **+ 2** `dork-list` entries, all `no_merge` so consolidation can't collapse a corpus into a technique page; each holds a capped excerpt while the full list is a **sidecar** under `data/kb/payloads/` , mounted read-only at `/payloads` in all three sandboxes so ffuf/wfuzz point straight at it. **56** `oscp_tools` files went to the Scripts Arsenal as file-backed rows (a path to copy, not 20,000 lines of PowerView), 38 Windows-only ones kept and marked `runs_here=false` (D9). Two env traps handled: rglob's OneDrive omission (→ union with `git ls-files`) and AV-locked files (→ git-blob recovery). KB 1,601 → 1,667; arsenal 1,028 → 1,137. The Defender exclusion on `HackPit\data` was added first.
- **Item 2 — evasion/OPSEC as an additive second channel (D10).** The blue-team detection view is **byte-for-byte unchanged** and still passes `assert_describes_not_prescribes` (a test proves every blue field is identical with and without the offensive half attached, and that the default `footprint()` carries no `opsec` key — so tagging/reports/run-annotation are untouched). `detection/catalog.py` gains a curated `OPSEC` table (10 notes keyed by spec key: what makes it loud, the quieter tradecraft, the tradeoff, and — mandatory — `still_recorded` , the honesty marker). `resolver.py` gains an opt-in `include_opsec` channel with its **own**

guard (`assert_opsec_is_separate`): a note that fabricates the honesty marker or advises disabling/clearing/tampering with a sensor is rejected. `report.py` gains an off-by-default red-team OPSEC roll-up. The panel renders it as a distinct amber section. LLM may extend to uncatalogued commands, `ai_suggested` .

- **Item 3 — HTTP repeater.** `cockpit/repeater.py` mirrors `:kali` containment (hardcoded open container, human-only + source-scan locked, audited) and adds a scope check `:kali` lacks. **Argv-only curl**, never a shell — method/headers/URL are discrete tokens, the body rides on stdin, so a header of `a; rm -rf /` is one literal `-H` . A human clicking Send *is* the approval (no per-send prompt); the orchestrator/agent have zero path to `send` . When the send names an active engagement the URL host is scope-checked (out-of-scope → 403, nothing sent). Frontend `/repeater` : compose/send, response view, per-session history with edit-to-resend and a line-level body diff. The VPS-for-callbacks piece stays its own decision (D2).
- **Item 4 — pivot/tunnel routing.** `cockpit/tunnels.py` — chisel/ligolo-ng lifecycle (start the listener in the engage sandbox, hand back the agent one-liner to paste on the compromised host; human-only start/stop, source-scan locked). `route_for` / `wrap_command` are **pure** — they compute a **visible** `proxychains -q` prefix (chisel/SOCKS) or leave the command unwrapped with a route note (ligolo/interface), applied *before* the approval screen so the human approves the exact argv (silent routing was rejected for exactly this reason). A tunnel's subnet enters scope only via `engagement.add_pivot_subnet` — the one deliberate, audited widening path; recon expansion still cannot widen. Frontend `/tunnels` : start form, live tunnels with the copy-ready one-liner + per-subnet "add to scope (by hand)", and a pure route preview.
- **Item 5 — exam-mode report templates + proof.txt as per-host state.** `state` `Host` gains `local_txt` / `proof_txt` + `ownership()` ; captured automatically when a command reads a flag file (`proof.txt` / `root.txt` → proof; `local.txt` / `user.txt` → local — a stray 32-hex hash is never mistaken for a flag) or pasted via `POST /sessions/{id}/state/proof` ; the state panel badges foothold vs owned. `report.py` gains four templates — standard, **OSCP** (per-host walkthrough + a proof.txt table spliced from state at `{{PROOF_TABLE}}` , computed not model-written), **CPTS** (exec-summary + findings register), **H1/Bugcrowd** (impact-first + a **CVSS 3.1** base score computed by `cvss31_base` , matching the official calculator). Every template keeps the shared grounding rules + the `{{EVIDENCE}}` splice. Frontend: a template picker on the report screen.

Phase 5 — the PTY gap, and finishing the KB

Commits `e8eeeef` , `8f0b91b` (terminal), `84e31bf` (CVE index), `f6fb6a4` (KB batches).

- **Item 1 — a real PTY, as a SECOND surface.** §1.1 used to call this a trade: readable transcripts *or* full-screen tooling. It is not, if they are separate surfaces. `:kali` is untouched — still sentinel-delimited over a plain pipe, still the clean per-command transcripts reports are built from — and `cockpit/terminal.py` adds `:terminal` beside it. `docker exec -t` refuses without a client TTY and a WebSocket handler has none, so the pty is allocated **inside** the container by a constant driver that `pty.fork()` s bash and multiplexes it over the pipes. Its stdin is **framed** (`0x00` = keystrokes, `0x01` = `COLSxROWS` → `ioctl TIOCSWINSZ`), which is what makes resize possible without an in-band escape a keystroke could forge; its stdout is the raw pty stream, straight to xterm.js. Containment mirrors `:kali` point for point: hardcoded container (the request carries `session_id` / `cols` / `rows` and nothing else), driver a raw-string constant passed as one argv element, no isolation gate, **human-only and source-scan locked** like `run_kali` , and the raw stream audited to the run store — capped, and checkpointed every 30s so a crash still leaves a transcript. The WS route pins `Origin` by hand, because CORS middleware does not cover WebSockets. `test_terminal.py` (16 checks) locks all of it *plus the additive property itself*: `kali.py` must still carry its sentinel and must never grow a pty, so the clean

transcript cannot quietly die. Verified live — `tty` reports `/dev/pts/0`, `top` and `vim` render, resize propagates 30×100 → 43×132 → 60×200.

- **Item 2 — CVE → exploit index (§3, finding 6).** The OSCP inner loop, built as a **keyed lookup rather than another KB batch**: "find the exploit for *this* version" wants an exact table, not prose retrieval, so it gets its own data shape, search path and surface. `pipeline/ingest_exploitdb.py` reads `/usr/share/exploitdb/files_exploits.csv` out of the sandbox image — 47,108 exploits, 27,384 with a CVE, 25,041 distinct — nothing fetched from the internet. exploit-db titles follow `<Product> <Version> – <Vuln>` on ~100% of rows, so parsing the left side turns a flat catalogue into `(product, version) → exploits` with the constraint typed (exact / lte / range / wildcard / none); 32,807 rows yield a version. `backend/exploits/` then does what `searchsploit`'s substring match cannot: `2.4.49` satisfies `< 2.4.50`, sits inside a range, and matches the `2.4.x` line — each with a *different stated verdict*, and the verdict is the ranking's primary key, with token similarity only breaking ties inside a tier. (Blending them let `Apache 2.4.x` outrank the entry naming 2.4.49 exactly — caught on real data, now regression-locked.) Surfaces: `/exploits`, plus a per-service `exploits →` link in the state panel, so a fingerprinted service reaches its exploits in one click. Every hit names the file **already inside the sandbox**. It executes nothing, and `test_exploits.py` asserts the package contains no subprocess path and never reaches the execution layer — finding is not running.
- **Item 3 — PortSwigger Web Security Academy (+372).** `pipeline/fetch_portswigger.py` turns the site into the same shape every other source has, so nothing downstream is special-cased. URLs come from the publisher's own `sitemap.xml` rather than from crawling — it is complete (the all-topics page renders client-side, so scraping it yields 19 of ~140) and nothing is ever guessed; `robots.txt` restricts only `/bappstore/bapps/download/`. 398 pages, 273 of them labs, whose *Solution* steps survive. Two rules written by what dry runs actually did: **labs and sub-topic pages are** `no_merge` (the first run folded ten Academy pages — Reflected/Stored/DOM XSS, contexts, CSP, dangling markup — into one pre-existing grab-bag called "xss resource", so "Reflected XSS" stopped being retrievable as itself; only a topic *root* may consolidate, which the Academy expresses as URL depth, taking 90 merges down to 26); and the two cheat sheets use `<section>` not `<main>`, so without a fallback they were 2 of 18 pages silently extracting zero bytes.
- **Item 4 — HackTricks Cloud (+578), and the cloud gap closed.** Same GitBook layout and adapter as the HackTricks book already ingested. Two corrections, both from dry runs: it is **not class-grouped** (grouping collapsed 44 distinct CI/CD pages — Jenkins RCE, GH Actions cache poisoning, Okta, Cloudflare — into one candidate), and `CANON` gains only **technique-level** cloud classes. "kubernetes", "ci-cd" and "supply-chain" name a *platform*, not a technique, and `CANON` is the authority on what consolidates; including them collapsed every "Kubernetes X" page into one entry. Their absence is now documented in `CANON` so they are not re-added. **cloud 2 → 535, supply-chain 1 → 47.**
- **Item 5 — the HTB Academy slice, audited (§3, finding 4).** Auditing what the local folder holds against what had been ingested found two files silently lost, both to one root cause — bare `rglob` over `*.md`. `File_Inclusion/README.md` (4.5 KB of LFI module content) is tracked in git but **gone from disk**, the dehydration/quarantine pattern this repo has hit before — and that module is full of web-shell examples, which is exactly what trips the signature. `File_Upload_Attacks/1.txt` (8 KB of module answers and webshell walkthroughs) was lost purely for its extension: §4.1's headline finding in miniature. Now `_all_md` (rglob ∪ `git ls-files`, with git recovery) plus `.txt`.

KB 1,667 → 2,617, 0 malformed rows, embeddings rebuilt incrementally, scripts index rebuilt, `entries.jsonl` verified present and counted after every pass (the Defender-quarantine trap).

Windows execution backend — the WinRM driver (shipped 2026-07-26)

Commits `72b0959` (transport + profiles + `windows` mode), `cf64b8f` (profile CRUD/picker + per-target `/tools`), `d8854dc` (native Windows abuse variants + AD live), `1ce53ce` (frontend), the VM guide, and this doc. Closes D9 the honest way. See `docs/WINDOWS-EXECUTION.md` + `docs/WINDOWS-TARGET-SETUP.md`.

- **A new execution transport, behind the same gates.** `docker exec` is swapped for a WinRM call; nothing about the safety model changes. A third mode, `windows` (alongside `lab` and `engagement`), is selected by `ExecRequest.windows_profile_id`. The target is the profile's **host — hardcoded server-side, never a request field** (the same containment shape `:kali` gets from its hardcoded container: a command physically cannot reach a box you did not pick). Gates: `windows` (profile exists) → `target` (host in the engagement scope, if one is also named) → **never-auto-run** approval → danger red-confirm. No isolation gate — it is a real external box, like engagement.
- **Model A — HackPit drives, it does not own.** No Windows VM lives inside the project. The operator runs a Windows/AD VM in VMware Workstation; HackPit opens a WinRM session and runs one PowerShell command string on the box (Rubeus/PowerView/Mimikatz/.NET all run *there*). `pywinrm` is **lazy-imported** so the hermetic suite needs no dependency and no network; the AD-live path is unit-tested with a **mocked** transport.
- **Saved connection profiles** (`cockpit/winprofiles.py`) — a "Windows targets" store in the gitignored `sessions.db`: name, host, transport (WinRM; SSH a documented later seam), port, username, `password` **or** `ntlm-hash` (pass-the-hash, presented `LM:NT`), domain. The secret is **write-only** — masked to `has_secret` in every view, read only by the transport, never in a response, record or command line. A **captured vault credential can fill a profile**, resolved server-side so the secret never round-trips.
- **The AD graph executes live.** Every abusable edge gained a **native Windows variant** (PowerView/Rubeus/Mimikatz) beside the Linux one; the walk/orchestrator's proposed command runs over WinRM when a Windows target is picked, and `advance` still requires an approved + exit-0 run. The danger heuristic learned the native destructive cmdlets (`Set-DomainUserPassword`, `Add-DomainObjectAcl`, `Set-DomainObjectOwner`, `Add-DomainGroupMember`, `Set-DomainRBCD`, `Invoke-Mimikatz`, …), so a native DCSync/password-reset trips the **same** red confirm as its Linux cousin — the oracle test now checks both transports.
- **Per-target tool reconciliation.** Windows-only tools (Rubeus/PowerView/Mimikatz/winPEAS) report runnable when a Windows target is selected and N/A on a Linux run — reconciled per active target (`/tools?windows_profile_id=…`), never a global flip. Served from `main.py` so the cockpit stays arsenal-blind.
- **Frontend.** A **Windows targets** page (`/windows`) — profile CRUD, masked secrets, connectivity test — and a "run on" picker in the AD walk (Linux sandbox vs a WinRM target); the confirm/flags/command shown follow whichever command will actually run.
- **Safety regression-lock** (`test_winrm_safety.py`): host-locked to the profile / no gate bypass / secrets never leak / **the orchestrator cannot auto-run WinRM** (transport reachable only from the gated executor + the human router probe, source-scanned). Functional path (`test_winrm.py`) uses a mocked transport.

Deferred to a live VM: the live/browser verification against a real Windows box, until the operator's VM is up (build + unit tests are hermetic) — the same way tunnels and AD-live execution were deferred.

Post-assessment refinements (2026-07-27)

Three items from a read-through of this assessment, each done the low-risk way. All query-time / config / ranking changes — **no KB re-ingest, no 15 MB rewrite, no Defender risk.**

- **KB consolidation audited (§4.2) — no re-ingest needed.** A read-only provenance audit confirmed the OSCP "3:1 collapse" is healthy dedup, not loss: 36 command-rich survivors, 12 folded into stronger canonical pages (still attributed), ~37 same-technique OS/lab splits merged, **0 dropped as low-value**. Also corrected the framing — `oscp-cpts-notes` is a tier-3 public GitBook repo, not the author's own writing. Decision: leave the KB untouched (re-adding with `no_merge` would recreate duplicate technique pages that hurt search).
- **Relevance-first KB ranking (§3, tiers).** The composer was already relevance-first (BM25 + cosine RRF; tier a small additive nudge), but a *thin* tier-1 stub still got a flat boost that could edge out richer, more relevant lower-tier content. Fixed in `pipeline/search.py`: the tier-1 boost is now **gated on substance** (a stub with no commands + little body gets nothing), and a **completeness nudge** (command-rich, any tier, saturating/capped) lets content decide close calls. Trust breaks ties among substantive entries; it no longer manufactures relevance. Locked by `test_search_ranking.py`.
- **SAST coverage widened (§ Code scan).** The offline Semgrep bundle grew from 19 rules (Python/JS/TS) to **34 across 8 languages** — Java/Go/PHP/Ruby/C# added (`rules/hackpit-languages.yaml`) — plus a **ruleset picker** (bundled / per-language, or a registry pack for the full online catalogue). Default scan loads the whole offline directory. Locked by `test_codescan_rules.py` (rules well-formed + 8-language coverage + `semgrep --validate`, resolved from the venv the runner uses so it validates rather than skips). The new rules were confirmed **valid** (semgrep `--validate`: 0 errors, 34 rules) and to **fire** on real vulnerable samples (Java/Go/Ruby/C#). One robustness fix fell out of that check: on some Windows builds semgrep itself *crashes* on a `.php` file, and the scan treated semgrep as fatal — so a crash now **degrades to a warning** (the scan still returns with whatever else ran, like bandit), instead of 502-ing the whole scan. *(Engagement export/import was considered and dropped — the existing report generator already covers a shareable dump.)*
- **UI cleanup (Frontend).** Small polish from a working session: the two shell tiles were merged to **one `:kali` tile that opens the real PTY** (`/terminal`) — full-screen tools render from the everyday shell, while the sentinel "transcript shell" (clean per-command records) is preserved off-nav at `/kali`; both backends and their containment tests stay intact. The **phishing** category is now surfaced on the front-page grid (its KB entries already existed; forensics/ics/supply-chain stay hidden). The **top nav** dropped the redundant `:library` tile (home is the wordmark), the search field was widened to a one-line field, the unused accent-swatch picker was removed (amber is the default and only accent), and the nav is **optically centered** (a small left nudge balances it against the heavier bordered search box, so it reads centered rather than geometrically dead-centre). All display-only; no backend touched.

reconFTW review + incorporation (2026-07-27)

A reconnaissance engine — reconFTW (six2dez, MIT) — was reviewed end to end for anything worth mining. The short version, and what was folded in:

What it is. An autonomous external-recon / bug-bounty runner: one Bash orchestrator plus eight modules driven by a config file, chaining ~100 tools end to end (optionally across a cloud fleet) and emitting a consolidated report. Its pipeline is OSINT → subdomain enumeration (passive → certificate transparency → ASN/CIDR → permutations → mass-resolve + wildcard-filter → NOERROR → web-metadata scraping → recursive → reverse-IP) → resolution/scope → host/port scan → web probe + screenshots → URL discovery + `gf` triage + JS mining + param discovery + fuzzing → vuln checks → report.

Bottom line: mine the knowledge, never the runner. reconFTW's mechanism — autonomous, fire-and-forget chaining — is the exact opposite of HackPit's one-gated-executor, one-approval-per-command model, so none of the runner, config engine, axiom fleet or monitor/notify machinery is adopted (see Part II's skip list). Its *value* is a

battle-tested set of recon tools, proven flag patterns, and a recon ordering — and that lands squarely on HackPit's thinnest area, external attack-surface discovery. So the worthwhile half was folded in as **data and grounding, not a new execution path**:

- **Arsenal (`backend/arsenal/tools.json`) — +28 tools, 73 → 101**, in a new `osint` category (6) plus `recon` (10), `web` (10) and `cloud` (2): the URL-triage set, permutation engines, mass-resolution, JS mining, ASN expansion, external OSINT, bucket enum and the injection completers, each with `<target>` -based invocation templates. `executes_nothing` is unchanged, and the arsenal safety suite (schema + inertness + no template hardcodes a host) passes at **101 tools / 257 templates** — a rendered invocation is still a string until a human approves it through the same executor.
- **KB methodology — two `checklist` entries** (`recon-methodology-*`): the subdomain-enum ordering and the URL→gf-triage→fuzz pipeline, folded in by a new additive ingester (`pipeline/ingest_recon_methodology.py`) that mirrors the corpora discipline — byte-preserving pass-through, its own idempotency marker, re-run byte-identical, and it segregates the corpus block to the tail so `ingest_corpora` 's byte-identity invariant still holds (verified — `test_corpora` green). KB **2,617 → 2,619**.
- **Composer grounding** — an advisory recon-ordering note in the real-target proposer prompt: the sequence as guidance, still one gated command at a time, never a chain.
- **Sandbox image (`docker/Dockerfile.sandbox`)** — the 28 tools installed the way each ships (`go install` for the Go set, venv + PATH wrapper for the Python set, apt for `massdns` / `commix`), with `gf` pattern packs and a static resolver list baked for the no-egress lab. `subwiz` is catalogued but **not baked** — it fetches an ML model at runtime, so it runs only in the egress-enabled sandboxes (engagement / `:kali`), never the airgapped lab; `gotator` / `regulator` cover permutations offline. The ~8–10 GB image was rebuilt and smoke-tested — **all 26 baked tools resolve** in the fresh `hackpit/kali-sandbox:m1` (trufflehog moved to its release-binary installer after a recent release began requiring Go ≥ 1.25; `subwiz` intentionally absent), with the `gf` pattern packs (37) and a 12,956-line resolver list baked in.

The methodology, the arsenal entries, the composer note and the image rebuild are all complete and verified — arsenal suites green at **101 tools / 257 templates**, `test_corpora` byte-identity intact, the full hermetic safety suite green, and the fresh image resolving every baked recon tool.

Persistence / backdoors (TA0003) enrich (2026-07-27)

The post-exploitation **persistence** layer was reviewed the same way — audit first, add only what is genuinely missing — because HackPit already *executes* persistence on scoped targets through the one gated executor (engagement + WinRM + `:kali`) and `msfconsole` is installed. This is an enrich: **knowledge, catalog and describe layers only — no new execution capability, no persistence engine or autorunner.**

What the audit found. The earlier "~60 persistence entries" figure conflated **cloud** persistence with host persistence: of the ~72 persistence-relevant KB entries, **52 are cloud** (IAM / AAD / GCP backdoors) and only **2** were dedicated host-persistence entries. The TA0003 *mechanisms* were present, but only at the command level, scattered through the OSCP/HackTricks corpus (scheduled-task/cron 64 hits, services 59, backdoor accounts 70, web shells 46) — what was missing was the **organised, per-mechanism methodology** tying them together, exactly the reconFTW shape (the raw material existed; the map did not). On the detection side, `attck.py` carried almost none of the persistence technique rows.

- **KB methodology — two `persistence` checklist entries** (`persistence-methodology-windows` , `persistence-methodology-linux`): a TA0003 mechanism map per OS — Registry Run keys, Startup folder, scheduled tasks, services, WMI subscriptions, accessibility/IFEO, backdoor accounts and web shells on Windows; cron, systemd

units/timers, SSH `authorized_keys`, shell-init profiles and backdoor accounts on Linux — each mechanism a single gated command plus the footprint it leaves, cross-referenced to the detection panel. Folded in by a new additive ingester (`pipeline/ingest_persistence_methodology.py`) mirroring the recon-methodology discipline (own idempotency marker, byte-preserving pass-through, corpus block segregated to the tail, re-run byte-identical). KB **2,619 → 2,621**; `test_corpora` byte-identity intact. Rootkits and bootkits are named as knowledge only, never tooling.

- **Arsenal (`backend/arsenal/tools.json`) — +4 tools, 101 → 105**, in a new `persistence` category: **SharPersist** (`platform:windows`, add/remove templates, mirroring `rubeus`) plus **Sliver**, **Empire** and **weevely** as **reference-only** — catalogued but not installed, so reconcile reports them not-present and the planner will not propose them (the same treatment as `subwiz`). Their install and C2 wiring are **deferred to build #4**. `executes_nothing` unchanged; arsenal suites green at **105 tools / 262 templates**.
- **Detection footprint (`backend/detection/`) — 13 `FootprintSpec`s + 11 ATT&CK rows.** `attck.py` gained the persistence technique rows (T1547.001, T1543.003/.002, T1053.003/.006, T1546.003/.004/.008, T1136.001, T1505.003, T1098.004) — additive, transcribed from ATT&CK v19.1 and trimmed to the Windows/Linux/network log-source subset, verified against live upstream. `catalog.py` gained a describe-side footprint per mechanism (Event IDs / Sysmon / Autoruns / auditd, loudness, and a `why_rating` that carries the honesty marker — *quiet = a defender coverage gap, not an operator advantage*). This is the **blue/describe half only**: no OPSEC/evasion notes were added (that channel is build #4), so the `assert_opsec_is_separate` and never-prescribe guards are untouched. Aliases were added for unambiguous binaries only (`schtasks`, `crontab`, `useradd`, `sharpersist`, `weevely`); multi-purpose `reg` / `sc` / `net` / `systemctl` stay on the argv path so a `reg query` is never mislabelled persistence.

All three landed additively and verified — the full hermetic safety suite green (`test_detection`, `test_detection_safety`, `test_arsenal`, `test_arsenal_safety`, `test_corpora`), and `pipeline/detection_sources.py --verify` clean against live ATT&CK v19.1 + SigmaHQ. No image rebuild (OS built-ins and the installed `msfconsole` cover the mechanisms), and no new execution path — the gated executor already ran these commands, one human approval at a time.

AV/EDR evasion + traffic obfuscation (2026-07-27)

Build #4 closes the C2/evasion channel the persistence enrich deferred. Unlike that enrich, this one **does add execution capability** — a Sliver C2 surface, DNS-tunnel listeners, and a generate-only evasion engine — so the whole design is about making the new surfaces inherit the existing containment rather than sit beside it. It also makes a **deliberate policy change**, recorded as D16 below.

D16 — the OPSEC channel may now prescribe evasion. Until this build, `assert_opsec_is_separate` refused any sensor-blinding phrasing outright: the channel could say "rate-limit and add jitter" but not "patch AMSI". That ban is **lifted entirely and deliberately**. The reason is coherence: this build ships an engine that *emits* AMSI-patch and ETW-blind artifacts, and a tool that produces an artifact it is forbidden to describe is worse than one that describes it honestly. **Two invariants survive and are now the whole contract:** every OPSEC note must carry a substantive `still_recorded` naming what catches the technique anyway, and the blue-view footprint is always produced alongside and is never suppressible. What did **not** change: the blue-side describe-never-prescribe guard (`_evasion_prescription`) is untouched, so the defender's copy is byte-identical to before; and the four execution gates — human approval per command, scope lock, red-confirm, audit — are untouched. This is a **posture shift on a public repo** and is stated plainly rather than buried: HackPit now documents in-process sensor tradecraft, always paired with its detections.

- **Sliver C2 (`backend/cockpit/sliver.py`) — a split-gated surface.** Server lifecycle (start/stop/list) is **human-only** with no red-confirm, mirroring `tunnels.py` : it is operator infrastructure on the operator's own sandbox, so clicking start *is* the approval. Implant **generation** is a **gated command**, mirroring `session.py` : it builds an `ExecRequest` and runs the real `executor.validate_request` — approval, scope, red-confirm — before anything is produced. It only generates; there is no delivery or execution route, and live beacon catch is deferred. `<listener>` is the operator's callback address and passes through verbatim, never target-substituted. A defect found while building it: `allowlist._FRAMEWORKS` held `"sliver"` , which never matches `sliver-client` , so an implant build would have passed the danger gate with **no red-confirm at all**; the real binary names were added.
- **DNS-tunnel obfuscation (`backend/cockpit/obfuscation.py`) — dnscat2 / iodine listeners**, entirely human-only. The far-side client half is returned as a **string for the operator to carry across by hand**; the module has no delivery primitive and must never gain one. The tunnel's pre-shared key is masked **at source** — `client_command` is built from a masked copy, so the real key never occupies a field that crosses the HTTP boundary. That mattered: the obvious fix (`model_dump(exclude={"secret"})`) still leaks, because the key is embedded verbatim inside the one-liner.
- **Bespoke evasion engine (`backend/evasion/`) — generate-only, with forced honesty.** A top-level package (peer to `detection/` and `state/` , per the cockpit/arsenal decoupling rule). It runs donut/ScareCrow as argv-only `docker exec` into a container resolved from the execution mode — never a request field — and **never runs or deploys what it builds**: the produced artifact is never `argv[0]` , and no deploy primitive exists in the package. **The honest half is computed *before* anything is built**: if the engine cannot resolve the blue-view footprint for a technique it refuses outright rather than emit a footprint-less artifact. Both the footprint and the `still_recorded` note are required fields on the result model, so no route can return one without the other, and the two PowerShell stubs carry the same honesty header *inside the artifact* so it survives being copied out of HackPit.
- **Detection (`backend/detection/`) — 7 new footprint specs and 20 new OPSEC notes.** TA0011 gained `c2_dns_tunnel` , `c2_malleable_profile` , `c2_jitter_beacon` and `c2_domain_fronting` (describe-only; domain fronting is documented as largely dead); TA0005/TA0112 gained `evasion_packed_loader` , `evasion_amsi_patch` and `evasion_etw_blind` , which are what the engine maps onto. The 13 persistence specs from build #3 were backfilled with per-mechanism OPSEC notes — 7 new notes alongside the new specs plus those 13 is where the 20 comes from. **SPECS 46 → 53, OPSEC notes 10 → 30** (both re-counted against the live catalog; an earlier draft of this bullet said "4 OPSEC notes", which contradicted its own 10 → 30). Two accuracy corrections were made under review: T1029 had been filed under TA0011 when *Scheduled Transfer* is TA0010 Exfiltration (it was dropped rather than retagged, since an Exfiltration label on a beaconing footprint reads as wrong), and the telemetry strings on five new rows had been written rather than transcribed — replaced with real ATT&CK v19.1 values, `--verify` clean against live upstream. No new ATT&CK rows were needed for the evasion specs: under v19 naming T1562.001 **and** T1562.006 (Indicator Blocking) *both* revoke into **T1685**, which was already present. An earlier version of this claim paired T1562.006 with T1690 and mapped the ETW footprint onto it — T1690 is the successor of T1562.003 (*Impair Command History Logging*) and covers shell history, not sensors. The Tasks 8–13 review caught it; the spec now cites T1685. `--verify` did **not** catch it and could not have: it checks that a cited id's name, tactic and log sources match upstream, not that the technique is the right one for the activity.
- **Arsenal — 105 → 110 tools**, new `c2` and `evasion` categories; Sliver moved out of `persistence` into `c2` . `<tunnel-zone>` and `<tunnel-net>` are new placeholders rather than reuses of `<domain>` , because the composer's target substitution rewrites any dotted token — spelling them `<domain>` would have pointed an operator's tunnel at the system under test. A latent data bug was found and fixed here: `placeholders`

declared `"<payload>"` **twice**, so every `json.load` silently discarded the first description; a duplicate-key check now guards it.

- **Image (`docker/Dockerfile.sandbox`) — Sliver, dnscat2, donut and ScareCrow installed; iodine was already present.** Three of the plan's assumptions did not survive contact with the base image and were caught by the layer's own smoke tests: `dnscat2` and `ScareCrow` are **not packaged** in kali-rolling, and there is **no Go toolchain** in the image, so release binaries and a from-source dnscat2 build were the only options. Two further faults surfaced only because the smoke test *invokes* each tool rather than running `command -v`: dnscat2's server dies on `require 'ecdsa'` (four gems missing), and `donut-shellcode` is a C extension with no console script, so `python -m donut` cannot work and a real CLI had to be written.

Containment, restated. No orchestrator, agent or loop module can reach any of the three surfaces — asserted by whole-tree source scans, not by convention. Nothing here is autonomous, no gate was weakened, and the only capability the agent gained is none: every new action is human-initiated and audited.

Gate-integrity audit (2026-07-27)

After build #4 shipped, the whole project's **guards** were audited with one question: *would this actually fire?* Not "is a guard present" — the build had already shown that a guard can be present, look correct, pass its tests, and never trigger. **24 guards were probed by constructing a deliberate violation for each. Seven silently failed to fire.** Full detail in `GATE-AUDIT-FINDINGS.md`; the three criticals were:

1. **Eight catalogued tools passed the danger gate with no red-confirm** — demonstrated end to end through `executor.validate_request`. `weevely` (which both generates a webshell *and* drives it), dnscat2's client and server, `iodine` / `iodined`, `commix`, `SSTImap`, `SharPersist` and `Invoke-Obfuscation` all returned zero reasons, as did the argument-driven `sqlmap --os-shell`, `commix --os-shell` and `netexec -x`. **FIXED and pushed (`4492fec`).**
2. **The WinRM transport classifies only the first token of what is a whole PowerShell script.** `executor.py` joins command+args and `run_ps()` executes the lot, so `;` and `|` are live separators: `Write-Host go ; Invoke-Mimikatz` is silent, while the same cmdlet as `argv[0]` fires. **FIXED — build #5 (see "Gate integrity — build #5" below).** This was the most consequential finding of the whole review cycle, because it executes on a real domain-joined host under real credentials.
3. **The `:kali` human-only lock never opens 39 of 69 backend modules** — its scan globs only `backend/*.py` and `cockpit/*.py`. A planted `from cockpit.kali import run_kali` in `adgraph/orchestrator.py` passes. The same narrow glob was copied into the tunnels, repeater, terminal and WinRM locks, and the cockpit→arsenal lock covers 5 of 22 modules. **FIXED — build #5 (see "Gate integrity — build #5" below).**

What Critical 1's fix actually changed, because the shape matters more than the list. The `.exe` / `.py` / `.ps1` normalisation that the AD sets already did was **collapsed into one shared normaliser used by every set** — that asymmetry was the root cause, not a symptom, and it was why `powershell.exe`, `nc.exe` and even `sliver-client.exe` produced no reason at all on the transport where `.exe` is the ordinary spelling. Two new sets were added rather than padding `_FRAMEWORKS`, so the reason the operator reads is true: `_TUNNEL_TOOLS` (a tunnel is a C2 path and an exfil path, not a payload generator) and `_RCE_TOOLS`. `_FRAMEWORKS`' `ligolo` entry was corrected to `ligolo-proxy` — the binary the repo actually runs — an entry that could never match, added *after* the `sliver` / `sliver-client` bug was fixed, and worse than no entry because it reads as coverage.

The finding underneath all three. `test_arsenal_safety` claimed to verify that a catalogued dangerous invocation demands the red-confirm — while testing `python3`, **a command that is not in the catalog at all.** That single choice is why eight tools shipped ungated. It now pins all **176** catalogued invocation names (every name,

alias and template `argv[0]`) into exactly one bucket, so an unclassified tool fails the suite; five planted regressions were run against it and all five fail correctly. Every critical in this audit, and two of the five findings in the Tasks 8–13 review, reduce to the same root cause: **a test that could not fail, or that tested a value the real system never produces.** That is what build #5's third task turns into a written repo convention.

Gate integrity — build #5 (2026-07-27)

The audit's two remaining criticals, and the testing pattern that hid them. Plan: `docs/superpowers/plans/2026-07-27-build5-gate-integrity.md`.

Critical 2 — the WinRM danger gate read the first token of a whole PowerShell script. `cockpit/executor.py` joins `command` + `args` into one string and the Windows transport hands that string to PowerShell, where `;`, `|` and newlines are live statement separators. The heuristic classified `basename(argv[0])` and scanned only the *args* for markers. So the gate was defeated by moving a cmdlet one token to the right, on the one path that executes against a real domain-joined host under real credentials:

Script	Before
`Write-Host go ; Invoke-Mimikatz -DumpCreds`	no reason, allowed
`Get-DomainUser | Set-DomainUserPassword`	no reason, allowed
`IEX (New-Object Net.WebClient).DownloadString(...)`	no reason, allowed
`Write-Host go` + newline + `Invoke-Mimikatz`	no reason, allowed
`powershell -enc <base64>`	fired **by accident** — `-enc` splits into `-e`, `-n`, `-c`

All five became tests first, verbatim, and were confirmed failing before anything was written.

The fix shape, and why it is not a parser. Four options were weighed; the choice was to **scan the whole joined string for every marker, wherever it appears**, and explicitly *not* attempt statement splitting. Correct PowerShell tokenising has to handle quoting, `$( )` subexpressions, the `&` call operator, backticks and line continuations — and **any parser written here becomes a new bypass surface, which is precisely the bug being fixed: the classifier's model of the input was narrower than the input.** Whole-string scanning has no "position" to exploit, and that is the property the first-token design lacked. The cost asymmetry settles it: this gate raises a **red-confirm, not a block**, so a false positive costs one click while a false negative runs Mimikatz on a domain controller.

Two things a name-based scan cannot see were closed alongside it. **Download cradles** — `IEX (New-Object Net.WebClient).DownloadString(...)` never spells the payload's name anywhere, so the *cradle* is the marker. **Encoded commands** — `-enc` defeats every text scan by construction, so the payload is decoded and re-scanned, matching every PowerShell prefix of the flag rather than only the spellings a human types; a blob that will not decode is itself reported rather than passed.

The root-cause lock. The bug was never only "the heuristic is too narrow". The string the gate classified and the string the transport executed were **built in two places**, and scanning a different string than the one that runs would have reproduced the bug somewhere new. Both now derive from `executor.join_ps_command()`, and a test asserts on the source — transitively, with a negative control — that they still do. The AD orchestrator's advisory pre-check was moved onto the same union, so the panel can no longer promise a confirm the gate would not require, or stay quiet where it would.

The false-positive cost, measured rather than promised (plan Task 1 Step 6). Run across every AD/Windows template in `tools.json` and every command in the AD graph's edge definitions: **78 of 145 invocations in the full population (53%), and 27 of 43 in the WinRM-reachable subset (62%), now demand a confirm.** The second number is the one that matters — the catalog's Linux impacket invocations go through the docker path, which is untouched. Of those 27, **none is a false positive**: every one is a genuine destructive abuse (Set-DomainUserPassword, Add-DomainObjectAcl, Add-DomainGroupMember, Invoke-Mimikatz, SharPersist, Invoke-Obfuscation, Rubeus ptt, Set-DomainObjectOwner). Read-only enumeration stays silent — Rubeus kerberoast and asreproast, `Get-DomainUser -SPN`, `Find-InterestingDomainAcl`, `Find-LocalAdminAccess`, winPEAS, `Get-ADServiceAccount`, `Get-DomainObject` — and that half is pinned as a control, because a banner on ordinary enumeration is how an operator learns to click through the one that matters. The measurement also found **two real false negatives** the sets had missed: `Invoke-SQLOSCmd` (OS command execution on a SQL host) and `Enter-PSSession` / `New-PSSession` (a further hop out of an existing session). Both were catalogued; both were silent.

Because whole-string scanning over-flags by design, **every reason names the marker it matched and the offset it matched at** — `powershell script: 'invoke-mimikatz' at offset 14 — dumps/ replicates domain credentials`. A reason the operator can evaluate is a gate; a banner they cannot is decoration, and trading a silent bypass for a decorative one is not a fix.

Critical 3 — eleven source-scan locks, nine of them narrower than their own docstrings. The `:kali` human-only lock is the entire reason the one deliberately-unbounded arbitrary-shell surface is considered safe. Its docstring said *scan the whole (non-venv) source tree*; it globbed `backend/*.py` plus `backend/cockpit/*.py`, which is **30 of 69 modules**, so all of `adgraph/`, `arsenal/`, `codescan/`, `detection/`, `evasion/`, `exploits/` and `state/` were never opened and a planted `from cockpit.kali import run_kali` in `adgraph/orchestrator.py` — a literal orchestrator module — shipped green. Three distinct defects, now fixed structurally in one shared `backend/test_support/scans.py` rather than by asking each suite to remember:

1. **Narrow file selection** — an `rglob` over the whole backend. Coverage per lock goes 30 to 67 modules; there is no per-suite glob left to get wrong.
2. **Basename allow-lists** — `{"router.py"}` matched against `f.name` exempted `adgraph/router.py`, `detection/router.py`, `arsenal/router.py` and every other `router.py` in the tree by accident. Allow-lists are now keyed on repo-relative POSIX paths only.
3. **Four literal substrings** — an AST pass alongside them catches what the audit planted and the old predicate missed: aliased imports, imports opened inside a function body, `import_module("cockpit." + "kali")`, `getattr(m, "run_" + "kali")`, and f-strings with constant parts.

Prose is stripped so a module that *documents* a rule does not violate it — this is not hypothetical, documenting the Critical 2 fix tripped the WinRM lock — but ordinary string literals are deliberately **kept**, because a scanner that blanked every string would go blind to `import_module("cockpit.kali")`, which is the indirection the lock exists to catch.

Two more locks were closed with it: the **cockpit→arsenal** lock checked a hardcoded list of five filenames while printing "the cockpit package has zero references to the arsenal" (the package has 22 modules, 17 of which post-date the invariant), and the **evasion agent-path** scan still filtered by *filename*, so `from evasion import engine` in `cockpit/executor.py`, `chat.py` or `adgraph/techniques.py` was never looked at. A module named something nobody predicted is exactly what a name filter cannot cover.

Widening the scans produced two false positives, and both were fixed in the predicate. `cockpit/reconcile.py` "referenced the arsenal" — as a *parameter name*; it takes the loaded catalog as an opaque injected object precisely so it has no import-time dependency. `detection/{catalog,resolver,router}.py` and `report.py` "reached the evasion engine" — they are the *anti*-evasion guard, the code that refuses prescriptive evasion copy. Both locks

now assert the claim the invariant actually makes (no import, in any form the AST can see). **Narrowing the file set to silence a false positive is how these guards got broken in the first place**, so that is written into the convention as a rule.

Task 3 — the convention, which is the part that outlasts both fixes. Every critical in this audit, and two of build #4's five review findings, reduce to one root cause: a test that could not fail, or that tested a value the real system never produces. `test_arsenal_safety` claimed to verify that a catalogued dangerous invocation demands the red-confirm while testing `python3`, a command absent from the catalog it was guarding — one choice that hid eight tools. `backend/AGENTS.md` now carries the rule, with the five failures that produced it in a table: **a safety test must iterate real data, assert on what it actually checked, and prove it can fail.** Concretely — draw inputs from the real source of truth so a tool or module added tomorrow is covered by nobody remembering; assert on the *filtered* count, never files opened; and carry a planted-violation positive control in the same test. `test_scans.py` runs **first** in the suite for that reason: ten locks rest on the shared scanner, and it must demonstrate it catches a planted violation before any of them means anything.

Sweeping for the *pattern* rather than the files the plan named found three more, none of which was in the task list. The **live-session stdin lock** — the load-bearing one, since anything able to type into an already-approved running session is executing un-gated commands — said "the whole source tree" and globbed three directories. `test_sliver_safety` and `test_obfuscation_safety`, the two suites the audit called correct, incremented their `scanned` counter *above* the `test_` skip, so their `>= 40` controls counted files opened rather than files judged. And re-running the audit's own probe list surfaced a real heuristic gap: `sekurlsa::pth ... /run:cmd.exe` as `argv[0]` fired only because `cmd.exe` happens to sit in its args (change it to `/run:powershell.exe` and it went silent), while `kerberos::list /export` — which writes every ticket in the session to disk — matched nothing at all.

A follow-on fix, after the plan's three tasks: the tunnel listener start is now gated (`I2`). It was outside build #5's plan and was recorded here as open, then closed immediately after. `cockpit/tunnels.start_tunnel` reached `subprocess.Popen` directly with no `validate_request` call, and `POST /cockpit/tunnels` carried no `approved` / `dangerous_ack` field — so a pivot listener, which is a C2 path and an exfil path in one, was raised on a plain POST with human-only as its *sole* bound, strictly less than every other execution surface gets. The docstring's claim that "the rewritten pivoted command still runs through the gated executor" was true and beside the point: the listener start *is* the tunnel primitive. Fixed test-first (the unapproved-start test was confirmed failing on the old code): `start_tunnel` now runs the real `executor.validate_request` before anything spawns, `TunnelStartRequest` carries `approved` + `dangerous_ack` defaulting **false** (an omitting client is refused, never silently granted), and the route maps a safety refusal to 403 naming the gate while an availability problem stays 409. The danger verdict comes from the server binary — `chisel` and `ligolo-proxy` are both in `_TUNNEL_TOOLS`, pinned by a positive control so the leg cannot pass vacuously — and a start now requires an engagement, because without one it would resolve to lab mode and fire the lab's isolation gate about a container the listener does not run in. The gated argv and the spawned argv derive from one `server_argv_for`, the same drift lock Critical 2 uses. This is deliberately **not** the D17 shape: for Sliver, C2 lifecycle is human-only while artifact generation is gated, because a listener nothing has connected to is inert; a pivot listener's whole purpose is to become a route into an unseen network, so it is gated like an execution.

What still remains open. `iter_run`'s prevalidated path re-checks approval but not danger (nothing is exposed today, because the single `prevalidated=True` caller validates first, but the asymmetry is the kind of thing a future caller trips over), and there is still no route-level authorisation on any of the routes, which remains the known localhost-only posture rather than a new finding.

Live fire, image pinning, and the prevalidated danger re-check — build #7 (2026-07-27)

Three items from Part II's deferred list, none of which adds offensive capability. One makes the image byte-reproducible, one closes a latent asymmetry in a belt-and-suspenders gate, and the largest — live-fire verification — went looking for evidence that the C2 and tunnel surfaces work and came back with three defects and a lot of confirmed containment.

Live fire: what ran, and what it found

`docker/proof/live_fire_proof.sh` builds a lab that is ours end to end — four containers on four `internal: true` networks (`docker/proof/live-fire-lab.yml`), no third-party target, no public callback — and drives the surfaces through their shipped entry points via `docker/proof/live_fire_driver.py`. Every gated step **proves the refusal first**: the same request with the acknowledgement missing must be refused with nothing spawned, because a proof that only shows the happy path cannot tell a working gate from an absent one. Results are reported as PASS / FAIL / **NOTRUN**, and a step that could not run is never folded into a pass. The run stands at **26 passed, 3 failed, 5 not-run**.

The gates and the containment held, live and without exception:

- **The `I2` pivot gate fires on all three limbs** against a real `ligolo-proxy` start — no engagement refuses at `gate=engagement`, unapproved at `gate=approval`, un-red-confirmed at `gate=danger` — and the approved start is accepted. This is the first time that gate has been exercised outside a stub.
- **The Sliver implant gates fire** — `gate=approval` unapproved, `gate=danger` without the red-confirm.
- **The pivot subnet enters scope only through the explicit amendment**, verified against a before/after snapshot of the engagement record, with the control taken first: the deep target is confirmed *unreachable* from the operator box before any tunnel exists, so "we reached it through the tunnel" would have meant something.
- **Containment holds live, measured from inside the contained host.** The implant host cannot reach the internet (`1.1.1.1`) or the host (`host.docker.internal`), and *can* reach the C2 — so the beacon has nowhere else it could go, which is what makes demonstrating a callback safe in the first place.
- **The DNS tunnel key never enters the pasteable one-liner.**
- **A real Sliver server runs.** `sliver-server daemon` starts through `cockpit/sliver.py`, unpacks its embedded assets and holds `:31337`.

Three things **failed**, and they are defects in the surfaces rather than in the gates. All three are the same shape: a claim that unit tests asserted structurally and nothing had ever executed.

1. **The Sliver implant build cannot succeed as shipped.** `_implant_argv()` builds `sliver-client generate --os … --mtls <listener>`, and `sliver-client` has no `generate` subcommand — it has `import`, `version`, `completion`, `help`. In Sliver 1.5 `generate` is an *interactive console* command. The existing tests assert the argv's **shape** (that `<listener>` is emitted verbatim, that the target is never emitted) and never that Sliver accepts it, so a build that could not run looked correct for as long as nobody ran one. The console route was verified working during this build with the identical flags over stdin — `echo "generate --os linux --arch amd64 --format exe --mtls H:P --save …" | sliver-client` produced a 14 MB implant in 27 s — so the fix is known and small. **It was deliberately not applied here**: making a non-functional offensive path functional is a capability change, and this build adds none. It is the operator's call.
2. **`tunnels.start_tunnel` reports `status="listening"` for a process that has already exited.** `ligolo-proxy` is an interactive console binary; spawned with no usable stdin it binds the port, prints `Listening on`

`0.0.0.0:11601`, reads EOF and exits 0. The status is assigned at `Popen` time and never observed, so the panel shows a live pivot over a dead process. Confirmed by `proc.poll()` returning `0` four seconds after the model claimed it was up, with nothing bound in the container.

3. `obfuscation.start_listener` **has the same defect** for `dnscat2-server`, for the same reason.

Because the implant never built and the listeners never survived, the three end-to-end claims those steps feed are recorded as **not-run, not as passed**: no beacon callback was caught, no proxychained traffic reached the deep subnet, and no dnscat2 session was established. The harness now asserts process liveness directly, so these three become regressions the moment they are fixed.

Two further items are **not-run because they need operator infrastructure**, and no amount of local wiring substitutes for either:

- **A delegated DNS zone.** Only the direct-to-server dnscat2 path can be exercised locally. A genuine delegated tunnel needs a domain the operator controls with an NS record pointed at the listener. "Demonstrated in lab" and "needs a real delegated zone" are different claims and are kept apart.
- **iodine**, which needs a TUN device at *both* ends plus that same zone; the engage sandbox has `/dev/net/tun`, the lab client container does not.

A side finding, recorded rather than acted on: `obfuscation.start_listener` justifies being human-only-and-ungated by citing "the identical reasoning the pivot-listener lifecycle uses" — and the pivot listener was tightened in build #5 (`I2`) to require engagement plus approval plus the red-confirm. The cited precedent no longer holds. Whether the DNS listener and the Sliver server start should follow it is a design decision, not a defect, and it is left open.

One environmental fact surfaced immediately and is worth writing down: the running stack was still on `hackpit/kali-sandbox:m1` built *before* build #4, so none of the C2 tooling existed in the sandbox the gated code execs into. The compose file rebuilds that tag from the current Dockerfile; nobody had re-run it with `--build`. The stack was realigned and both network proofs re-verified afterwards — `isolation_proof.sh` 4/4, `engage_open_proof.sh` 4/4.

The image is now byte-reproducible — and its smoke test had never passed

`docker/pin-build4-versions.sh` resolved the real upstream values and rewrote the layer: dnscat2 to commit `42f8d783…` (it has no release tags, so a SHA is the only thing there is to pin to), the four Ruby gems to `trollop:2.9.10 salsa20:0.1.3 sha3:2.2.4 ecdsa:1.2.0`, and `donut-shellcode==1.1`. Two bugs in that script had to be fixed before it could run: it required `python3` on `PATH` (only the backend venv exists here) and its verification step used `grep … && { exit 1; }`, which under `set -e` exits 1 on the *success* path. `test_evasion.py` now asserts the stronger claim — all three installs pinned, three `ARG` version pins, the SHA a real 40-char object id, no floating form left behind — instead of tolerating a documented gap.

The fresh build then failed, and not because of the pins. The consolidated smoke test asserted `sliver-server version 2>&1 | grep -qi sliver`, and `sliver-server version` prints exactly `v1.5.42 - <sha>` — no substring "sliver". **That assertion could never pass, so this layer had never built since the check was added**; the image in the daemon predates it. The static guard that watches this block (`test_evasion.py`) reads the *shape* of these lines — `|| true`, `| head` — and by construction cannot know whether a grep pattern matches anything, so only a real build could surface it. Both sliver checks now match `v${SLIVER_VERSION}`, which is strictly stronger: it fails when the binary is missing, when it dies, and when it is not the version pinned above. `hackpit-kali:build7` builds clean, and all three pinned tools resolve and smoke-test by their real invoked names, with the shipped versions confirmed inside the image against the pins.

The prevalidated path now re-checks danger

`iter_run(prevalidated=True)` skips `validate_request` because the router already ran it to decide the HTTP status. It has re-checked **approval** for engagement and windows since build #4 — never-auto-run being the sole floor on a real target — but had no equivalent **danger** re-check. Nothing was exposed: the one `prevalidated=True` caller validates first. But "no caller does this today" is a coincidence, not an invariant, and the asymmetry was a trap set for the second caller.

Test-first per `backend/AGENTS.md`: the failing test showed a dangerous, un-acked, prevalidated request reaching `start` and spawning. The re-check mirrors the approval one and uses the **same per-mode classifier** `validate_request` uses — `windows_danger_reasons` on the joined script for windows, `dangerous_command_heuristic` on the argv for lab and engagement. The windows probe is deliberately a command only the script classifier catches (`Write-Host go ; Invoke-Mimikatz`, the Critical-2 shape), so a re-check wired to the generic heuristic fails the suite rather than passing quietly.

The refactor that made this work is worth recording because the project's own guard caught it. Routing all three validators through one shared `danger_reasons_for_mode()` dispatcher looked strictly better — divergence becomes structurally impossible — but `test_winrm_safety` asserts over the **static call graph** that `_validate_lab` can never reach `join_ps_command`, and that assertion is the positive control proving the reachability check can fail at all. The dispatcher reaches the PowerShell derivation on its windows branch, so wiring the docker-path validators through it emptied the control while every test still passed. The validators call their classifier directly and deliberately; only `iter_run`, which knows a `mode` string rather than a branch, uses the dispatcher. The classifier each side calls is identical either way — only the call graph differs, and the call graph is what the control reads.

One existing test needed updating rather than the guard being loosened: `test_winrm_safety`'s host-lock case drove `Invoke-Command` prevalidated with no ack. `Invoke-Command` *is* flagged, so the router would have demanded the red-confirm before ever setting `prevalidated=True`; the test had been relying on the gap it was sitting next to. It now carries the ack, which is what the real path sends.

The three live-fire defects, fixed — and a stale precedent decided (2026-07-28)

Build #7 found three surfaces that did not work as shipped and deliberately left them alone, on the grounds that making a non-functional offensive path functional is a capability change and that call belongs to the operator. The call was made: fix all three, and decide the design question the run had left open. This section is what that took, including two further defects the fixing surfaced — both of the same family, and both found the same way.

The Sliver implant build: a subcommand that does not exist

`_implant_argv()` built `sliver-client generate …`, and neither Sliver binary has a `generate` subcommand — `sliver-client --help` offers `completion`, `help`, `import`, `version`, and nothing else. In Sliver 1.5 `generate` belongs to the interactive **console**, which `sliver-server` hosts when run with no subcommand. A build now runs `docker exec -i <container> sliver-server` — still an argv **list** — and writes one `generate …` line to its stdin, followed by `exit`. Verified against the pinned 1.5.42 image: 16 s, a 14 MB implant on disk.

Two consequences follow, and both are handled rather than hoped about.

The console line is **text** reaching Sliver's own flag parser, so it is a parser this code did not previously feed. Everything interpolated into it is bounded before it arrives: `os` / `arch` / `format` / `transport` already came from fixed sets, `listener` is now pinned by a pattern that admits a host or IP with an optional port (or an angle-bracket placeholder) and rejects whitespace and a leading `-`, and the server-chosen artifact path is checked

whitespace-free rather than trusted — it is derived from a caller-supplied engagement id, and a space in it would split into extra console tokens. Nothing here reaches a shell: the argv is still a list, so `/bin/sh` is not in the picture at all.

And **the console exits 0 whether or not the build inside it worked**, so an exit code cannot decide success. `generated` versus `failed` is now decided by reading the artifact back out of the container with `stat`. That is a stricter test than the one it replaces, in both directions: a console that exits 0 having written nothing is `failed`, and a console that complains while producing a real artifact is `generated`. The gate surface moved with the binary — `_gate_request` classifies `argv[0]` itself rather than a constant, so the audit row names the process that actually runs.

Both listener lifecycles: a status that was assigned, never observed

`ligolo-proxy` and `dnscat2-server` are interactive consoles. `docker exec` **without** `-i` hands a container process a closed stdin no matter what the host process got, so both printed their banner, read EOF and exited — while `start_tunnel` and `start_listener` returned `status="listening"`, assigned at `Popen` time and never looked at again. The operator was then holding an agent one-liner for a port with nothing behind it.

Both halves are fixed in one place, `backend/cockpit/lifecycle.py`, now shared by all three lifecycle surfaces:

- **A console binary gets a stdin that does not end.** `-i` is per-binary and off by default — `ligolo-proxy` and `dnscat2-server` need it; `chisel server`, `iodined -f` and `sliver-server daemon` are real daemons and are spawned without one, verified rather than assumed.
- **The write end is a raw file descriptor, deliberately.** Passing an int fd rather than `subprocess.PIPE` leaves `proc.stdin` as `None`, so there is no writer object for any caller to reach for. Holding a listener's stdin open is a liveness requirement; being able to *type into* a live C2 console is a capability, and this grants the first without the second. A whole-tree source scan with a planted-violation control keeps the handle inside the four modules that own a listener.
- **The status is observed.** After a settle window: a process that exited is a **refusal** carrying the output it died on; a live process whose port `ss` confirms bound inside the container is `listening`; a live process whose bind could not be confirmed is `starting`. The probe is tri-state on purpose — "could not probe" is not "not bound", and neither is ever rendered as a confirmed listener. Every model now carries a `liveness` field holding the evidence, not just the verdict.
- **Output is drained** into a bounded ring buffer, so a chatty server cannot fill the 64 KB pipe buffer and wedge mid-engagement looking exactly like a hung tunnel.

The stale precedent, decided: both listeners are now gated

`obfuscation.start_listener` justified being ungated by citing "the identical reasoning the pivot-listener lifecycle uses" — and build #5's `I2` finding had already tightened that pivot listener to require engagement plus approval plus the red-confirm. The citation was stale. The larger problem was that `I2`'s actual argument applies to a DNS tunnel with full force: *a listener whose purpose is to become a route into a network the scope gate has never seen is an execution, not a lifecycle no-op.* A DNS-tunnel listener is the server end of a covert exfil channel — carrying arbitrary traffic out of a network through that network's own resolvers is the thing egress controls exist to stop. "No target" was true and beside the point, exactly as it was for the pivot listener.

The DNS listener is now gated the same way: engagement, approval, red-confirm, through the real `executor.validate_request`. `dnscat2-server` and `iodined` were already in the covert-tunnel danger set, so the heuristic fires on the binary and the red-confirm is always required.

The Sliver server start is gated too, on a weaker argument that is worth stating honestly. A Sliver daemon is genuinely less exposed than either listener: starting it opens no channel toward anything under test, it binds a multiplayer port on the operator's own sandbox for the operator's own console, and the steps that *do* reach a target — creating a listener, generating an implant, delivering it — are separate and separately gated. That argument is real, and it is not quite sufficient, because Sliver's server config can **persist listener jobs** that come back up with the daemon. "Starting it is inert" is therefore a property of a config file, not a property of this code, and a gate should not rest on something the module cannot see.

Human-only remains the load-bearing property for all three surfaces and is untouched: the whole-tree source scans still prove no orchestrator, agent or loop path reaches any of them. The gates are belt to those suspenders. What still differs between the two halves is the **target**: a C2 server and a DNS listener have none, so neither carries a target-lock, while implant generation is scope-checked as before — and `SliverServerRequest` is asserted to have no `target` field, because one would mean a scope gate firing on the operator's own box.

Two further defects, found the same way

Re-running the live fire against the fixed surfaces surfaced two more, both invisible to every unit test and both the same shape as the originals: a state reported rather than observed.

A stop did not stop. Killing a `docker exec` client does not kill the process it started inside the container. After stops that reported `status="down"`, `ss` inside the sandbox still showed `iodined` holding UDP/53 and two `chisel server` processes listening — and the next start failed with `EADDRINUSE`, a refusal about a listener the operator had been told was already stopped. A stop now closes stdin (which is enough for a console binary), kills the client, and then reaps the server inside the container by **its own argv**: `pkill -f` against the full command line, which carries the listener's port, so it stops that listener rather than every process of the same kind. A blanket `pkill -f chisel` would have stopped somebody else's tunnel.

`proxychains` pointed at Tor. Debian ships `/etc/proxychains4.conf` configured for `socks4 127.0.0.1 9050`, while HackPit's chisel plan brings up a reverse SOCKS5 at `127.0.0.1:1080` and `wrap_command` rewrites a command to `proxychains -q …` on the strength of it. With the stock config the rewritten command the operator approves would have quietly tried Tor and timed out. The Dockerfile now replaces that line rather than appending to it — proxychains reads the last `[ProxyList]` section, and leaving 9050 in place would make the chain depend on ordering. This is exactly the class of gap a proof harness exists to find: every unit test passed, and the shipped path could not work.

Both fixes verified against a rebuilt image, not just against a diff

Two of the five fixes carried a claim that had not itself been executed, which is the exact failure mode this build exists to eliminate, so both were closed properly rather than left as edits.

The `proxychains` line had never been built. The fix was written into `Dockerfile.sandbox` while the running image predated it, and the harness applied the same one-line edit at runtime — correct for making the proof valid, but it left the Dockerfile layer itself unproven. `hackpit-kali:build7b` now builds from the current Dockerfile with exit 0, and the config inside the image reads `socks5 127.0.0.1 1080` with **zero** `socks4` lines remaining. The stack was rebuilt and recreated from that Dockerfile, and the live fire re-run against it takes the "*already points at the chisel SOCKS5 — baked into the image*" branch rather than the runtime-patch branch, with the proxychained request still routing. The runtime patch stays in the harness on purpose, so the proof remains valid on an older image, and it says which branch it took.

The stop-reap was unit-tested but never demonstrated against the failure that motivated it. Run live: a `chisel server` started through the lifecycle and confirmed bound; killing only the `docker exec` client left the process running and the port still bound — the defect reproducing exactly as described; the reap then cleared both. And it is targeted, which is the part that matters: reaping one tunnel left a second `chisel server` on a different port still bound, where a blanket `pkill -f chisel` would have stopped it too.

That second run also surfaced a sixth defect of the same family, now fixed. On a freshly recreated container `ligolo-proxy` took longer than the settle window to bind, so it was honestly reported `starting` — and `route_for` correctly refuses to route through an unconfirmed bind. But the status was **observed once and never re-observed upward**: the list functions only ever demoted to `down`, so a listener that bound a second after its window closed stayed `starting` for its whole life, and therefore permanently unroutable. The refresh now moves in both directions and only on evidence — a dead process becomes `down`, a `starting` listener whose port has since appeared becomes `listening`, and a probe that says "not bound" or "could not run" promotes nothing. Only `starting` listeners are re-probed, so a confirmed bind does not cost a `docker exec` on every poll of the panel. The negative controls are in the same test, because a refresh that promoted on an unknown would be laundering exactly the uncertainty the settle-window observation exists to preserve.

What the live fire now shows

66 passed, 0 failed, 3 not-run (after the not-run-closing pass below). The defects are fixed and their assertions pass; the gates and the containment continue to hold, and iodine now demonstrates the DNS-tunnel class end to end.

- **A real implant builds** through the gated path — `status='generated'` with a 14 MB artifact read back out of the container by size — and **the beacon calls back**: the implant, run on a lab host whose only network is internal, registers a session with the operator's server. Containment is measured from inside that host: no internet, no Docker host, C2 only.
- **A proxychained request routes through the pivot** to a host on a subnet the operator box has no route to, returning the deep target's known body — and the argv that ran is the one `wrap_command` produced, so what was demonstrated is what a human would have approved. The same tunnel cannot reach `1.1.1.1`.
- **All three lifecycle gates refuse on all three limbs** — no engagement, unapproved, un-red-confirmed — with nothing spawned, for the Sliver server, both pivot kinds and the DNS listener. Each surface's status is separately asserted to have been *observed*, against the process handle and against the `liveness` evidence.
- **Every listener stays up** and its port stays bound for the whole phase.

Four things are **not-run, and they are not passes**:

- **ligolo's interface routing.** ligolo routes at the interface, and creating that route means typing `session` then `start` into the proxy's console. HackPit holds that console's stdin open and deliberately never types into it — an unapproved command on a live C2 console is precisely what the gates exist to prevent. Completing a ligolo session stays the operator's own step. The routed-traffic claim is carried by chisel, whose server needs no console at all, and that is why both kinds are exercised.
- **The dnscat2 client handshake** — now diagnosed decisively (see the next subsection): `tcpdump` shows the `dnscat2-server` process itself answering `NXDOMAIN` to correctly-formatted queries, reproduced with both ends in one container over loopback, which isolates it to the pinned dnscat2 build rather than to HackPit or the lab. The listener HackPit starts is gated, bound and stable throughout.
- **A delegated DNS zone.** Unchanged and unfakeable: a genuine delegated tunnel needs a domain the operator controls with an NS record pointed at the listener. "Demonstrated in lab" and "needs a real delegated zone" stay different claims.

- **A delegated DNS zone** — the public-hierarchy hop in front of either tunnel. iodine's IP-over-DNS channel itself is now **demonstrated** (below), carrying traffic and confirmed DNS-encapsulated on the wire; what stays not-run is only the delegation in front of it, which needs a domain and a public listener the operator does not have.

One harness lesson is worth recording, because the harness was briefly lying about the product. A console listener's lifetime is tied to the process holding its stdin — in the backend that is the long-lived server process, which is correct. But each driver invocation was its own short-lived Python process, so a listener came up, was confirmed bound by `ss`, and died the moment the driver exited, after which the shell's follow-up check reported "did not reach LISTEN" for something that had demonstrably been listening seconds earlier. Client connect-backs now happen inside the same process that holds the listener, so every assertion is measured while the thing it is about is alive. Three further false findings had the same character and are documented in place: a `pkill` sweep that killed its own wrapper shell on the first pattern and silently abandoned the rest, a `sed` whose absolute path Git Bash rewrote into a Windows path so the config edit never applied, and a `dnscat2` client invocation refused for passing both a `--dns` argument and a bare zone. A proof that reports its own bugs as product defects is worse than no proof.

Closing the not-runs: iodine demonstrated, dnscat2 diagnosed (2026-07-28)

Four things were left not-run after the surfaces were fixed. Two are now resolved — one by demonstration, one by a decisive diagnosis — and the remaining two are stated for what they actually are rather than carried as vague gaps.

iodine's IP-over-DNS tunnel is now demonstrated carrying traffic, through the gated surface. A dedicated lab fixture (`iodine-client` in `live-fire-lab.yml`) carries the one thing iodine genuinely needs and the dnscat2 host must not have — a `/dev/net/tun` device, `NET_ADMIN`, and root, because iodine brings up a whole tunnel *interface* and a `cap_add` is not effective for a non-root process. The listener still comes up through HackPit's gated `start_listener(kind="iodine")`, refusing on all three limbs first; iodine is the other tool on the same gated path, not a bypass. The client is then forced onto the DNS-encapsulated path with `-r` — load-bearing, because on a flat lab bridge iodine will happily fall back to raw UDP and report a "tunnel" that never touched DNS, which would be a dishonest pass. The proof confirms the real thing on the wire: four ICMP echoes reach the server's tun endpoint through the tunnel with 0% loss, and a `tcpdump` on the server's UDP/53 shows the traffic arriving as DNS-encoded queries under the tunnel zone — eight of them for that ping. That is a genuine interface-level DNS tunnel, gated, carrying traffic, confirmed DNS-mode. (One harness bug surfaced and was fixed in the same pass, and it is the honesty guard earning its place: `tcpdump` line-buffers, so a first draft grepped the capture file before the packets had flushed and reported "no DNS packets" for a tunnel that was demonstrably DNS-encapsulated. The check now waits out the capture window.)

dnscat2's failure is now diagnosed decisively, and it is not HackPit's. A `tcpdump` on the server shows the client's encoded queries arriving correctly formatted — `<data>.lab.hackpit.internal`, alternating MX/TXT/CNAME, which is dnscat2's own protocol — and the `dnscat2-server` process itself answering `NXDOMAIN` to every one. Put both ends in the *same container* talking over `127.0.0.1` — no network, no resolver, no HackPit code, the server's DNS driver confirmed started (`New window created: dns1`) — and it still `NXDOMAIN`s, in every client shape. That isolates it fully to the pinned dnscat2 build (commit `42f8d783…`): a defect in the tool's own client/server handshake, sitting entirely above the listener HackPit starts, which is proven gated, bound and stable throughout. It is not chased further — fixing upstream dnscat2 is out of scope, and iodine now demonstrates the DNS-tunnel class end to end regardless. The proof records this as a precise finding rather than a bare "did not work".

The two that remain are genuinely operator-side, and cannot be faked into a pass. A *delegated* DNS zone — the public-hierarchy hop in front of either tunnel — needs a domain the operator controls with an NS record pointed at the listener, plus a publicly reachable listener, i.e. the same VPS gap as D2. The operator has neither, so it stays recorded as a prerequisite. And ligolo's *interface* route is a deliberate **won't-do**, not a not-yet: completing a ligolo session means typing `session` then `start` into a live C2 console, and the only way HackPit could do that is a surface that writes to a listener's stdin — precisely the capability `test_lifecycle_safety` exists to forbid (the held stdin is a raw fd specifically so no writer object exists). chisel already carries the routed-traffic claim end to end, so nothing is lost by leaving ligolo's console step to the operator.

Build #8 — the reasoning copilot, and a measured substrate (2026-07-28)

Two things this build set out to establish, one about the agent and one about the ground it stands on: make the orchestrator a genuinely *deeper* proposer without giving it one inch more autonomy, and turn "110 tools" from a claim into a measured number.

The substrate actually runs, and here is the number

`reconcile.py` has long answered *installed?* with a `command -v` sweep. That is a weaker claim than it sounds: a binary can resolve on PATH and then die the moment the dropped capability is needed. So `backend/substrate_probe.py` asks the harder question directly — it invokes every catalogued tool in the **live sandbox image, under its real profile** (`no-new-privileges` + `CapDrop: ALL`) with a trivial call (`--version` / `--help`) and records three tiers: *catalogued → installed → actually-runs*.

97 of 104 catalogued Linux tools actually execute (93.3%), measured, not asserted. The breakdown is reported honestly, because a tool that does not run is never counted as covered:

- **2 installed-but-do-not-run** — `amass` and Empire (`powershell-empire`). Both are the documented landmine class, though not the one expected: their packaged wrappers shell out to `sudo` , which `no-new-privileges` refuses, so the binary resolves and then fails. (nmap, the *expected* casualty, runs fine — the image was hardened since that trap was recorded.)
- **5 not-installed** — `prowler` , `scoutsuite` , `kube-hunter` (cloud-audit tooling), `ghidra` , `subwiz` . Genuinely absent from the image; reported as gaps, not covered.
- **6 windows-only** — Rubeus/PowerView/Mimikatz/winPEAS-class entries, not applicable to a Linux sandbox by construction (D9).

The probe earned its place immediately: it surfaced two real catalog defects of exactly the kind Task 1 warns about — **a package that installs under a different binary name than the catalog uses.** Sliver resolves as `sliver-server` / `sliver-client` and Empire as `powershell-empire` ; the catalog listed neither alias, so both read as "not installed" until the aliases were added — and adding them then forced those binaries through the arsenal danger gate under their *real* invoked names (`sliver-server` , `sliver-client` , `powershell-empire` now each demand the red-confirm, the same `sliver` -vs- `sliver-client` bug the gate audit fixed once already, caught again by the same test). The full per-tool report is committed at `docs/substrate-coverage.md` (+ `.json`); the static half (catalogued names vs. Dockerfile install lines) is unit-tested and needs no stack.

The reasoning proposer (2.1–2.8) — deeper proposals, not autonomy

The orchestrator used to pattern-match: it saw a port and reached for the tool it always reached for. `backend/reasoning/` makes it reason, and every component is additive, independently tested, and produces **proposals + rationale only**:

- **2.1 working memory — a tried/failed ledger.** The single biggest anti-looping lever. The proposer is fed the full state plus a ledger of what has been tried and how it turned out (read from the exit code *and* the output — a tool can exit 0 having failed), and a lead ruled out by the critic is persisted as a dead end. A failed lead does not get re-proposed. Validated live: with a `curl` to a closed port recorded failed, the model proposed the SQLi step against the real finding instead of repeating the dead lead.
- **2.2 hypothesis-first proposals.** The schema now leads with `hypothesis` + `expected_signal`, then the command — enumerate-before-exploit as a shape, and a wrong guess made visible. A proposal missing hypothesis, expected_signal, or a citation **fails a validation gate** (invariant 3, expressed as code with a control that shows it can fail).
- **2.3 a candidate frontier, not a greedy single step.** The proposer surfaces N leads, each scored by *evidence strength* × *expected payoff*; the top is pursued and the rest are **persisted as untried leads**, so a dead-end recovers to the next-best instead of looping. This generalizes the AD graph's edge-frontier to the whole engagement. It is a set of leads, **never a run queue** — nothing in it fires.
- **2.4 failure diagnosis.** A connection-refused output yields a reachability check *before* another attempt, not a blind repeat.
- **2.5 a skeptic/critic pass.** A second look tries to *refute* the proposal against the evidence, reusing the CVE→exploit index's rule that **the version verdict outranks token similarity**: a proposal citing a CVE whose exploit targets Apache 2.4.49 is caught and downranked against an observed 2.4.58, no matter how well the words match. This is the check that kills the top hard-box failure mode — confident hallucination. The critic is **advisory only — it downranks and flags, it never suppresses**: a refuted proposal is surfaced with its concerns and a lowered confidence so the human sees it is shaky, but the proposer stays free to raise it again, and nothing is ever blocked from being run by hand. (The first live run against Ollama found a real bug in this pass — it read an IP octet as a port — which was fixed and regression-locked; the honesty guard earning its place.)
- **2.6 domain-specialist routing.** A web finding routes to a web lens, an AD state to an AD lens, a foothold to a privesc lens — instead of one generalist voice.
- **2.7 fingerprint-keyed retrieval.** An exact service+version fingerprint (case-based: "this stack was solved by X") ranks ahead of generic token matches. The plumbing is built now; **growing the exploitation-write-up corpus it keys into is a follow-on** — noted, not faked: on today's KB the ranker degrades to the base order.
- **2.8 model-tier routing.** A config lever, default OFF and inert: with no `reasoning_tiers` configured, every step uses the base config byte-for-byte; configured, a hard step can be routed to a more capable model.

The honest ceiling. Reasoning depth is mostly the base model. A small local model caps it no matter how good the scaffolding is — the same live run that produced a fully-cited, correctly-routed proposal produced, on a re-run, one with no hypothesis at all (which the schema gate then flagged, correctly). This build moves the loop *up the curve* on easy and known-pattern-hard boxes; it does not turn a 9B local model into something that "solves anything hard." What it reliably changes is the loop's behaviour: it stops re-proposing dead leads, it states what it is testing, and it refuses to cite a CVE that does not fit the version in front of it.

Web-exploitation and privesc depth (Tasks 3 & 4, still human-fires)

Two drafting surfaces, both propose/ground/generate only:

- **Web (`reasoning/webexploit.py`, `POST /sessions/{id}/webexploit/draft`).** Given a web finding in state, HackPit drafts the actual exploit — the sqlmap invocation, the SSRF metadata probe, the IDOR replay, the parameter-pollution request, the LFI traversal — with an explanation grounded in the bug-bounty KB and citations back to the finding, handed to the human to fire through the **existing** repeater/executor. Nothing in the path sends without the human.

- **Privesc (`reasoning/privesc.py` , `POST /sessions/{id}/privesc/ingest`).** Paste linpeas/winpeas output; it identifies the vectors (NOPASSWD sudo, SUID GTFOBins, PwnKit, capabilities, SeImpersonate, AlwaysInstallElevated), drafts the escalation for the strongest one grounded in KB + state, and hands it over. Execution stays human-approved.

What is re-locked, unchanged

The whole point is that the proposer got smarter and nothing else moved. `test_loop.py` 's source-scan lock is **extended onto the shared scanner across the entire `reasoning/` package**: orchestrator.py plus all eleven reasoning modules are proven to have no execution path — no subprocess, no executor run method, no `:kali` /sandbox — by AST call-analysis (so a *drafted payload string* that contains `os.system(...)` as exploit text is correctly **not** a violation, while a real call is), with positive controls. A companion scan proves there is **no auto-approve / batch-approve / run-the-chain** anywhere in the proposer path or the frontier. The four execution gates (scope, approval, danger red-confirm, isolation) are untouched and still fire.

KB exploitation-writeup corpus — the case-based material 2.7 keys into (follow-on, now landed)

Build #8 shipped 2.7's fingerprint retrieval as plumbing and flagged the corpus it queries into as a follow-on; that corpus now exists. `pipeline/ingest_exploitation_writeups.py` seeds **78 distilled exploitation writeups** keyed by service+version fingerprint — the initial 16 (vsftpd 2.3.4, ProFTPD mod_copy, Samba usermap, SMBv1/EternalBlue, Apache 2.4.49/.50 traversal→RCE, Shellshock, Jenkins, Tomcat, Drupalgeddon2, GitLab ExifTool, Log4Shell, unauth Redis, BlueKeep, Kerberoasting, AD CS ESC1, Zerologon) plus a second batch of 19 (Confluence OGNL, Struts2, Citrix, F5 BIG-IP, PrintNightmare, ProxyShell, ProxyLogon, Spring4Shell, FortiOS, PHP-CGI, Grafana LFI, WebLogic, phpMyAdmin, MSSQL xp_cmdshell, SNMP, PostgreSQL, Webmin, Joomla, Elasticsearch), and a third of 43 across four themes — **AD depth** (AS-REP roasting, DCSync, noPac, unconstrained/RBCD delegation, PetitPotam+ESC8, GPP cPassword, LAPS, pass-the-hash, golden ticket), **unauth network services** (MongoDB, Memcached, CouchDB, MySQL UDF, JMX/RMI, IPMI, VNC, SSH user-enum, anon FTP/LDAP, SMTP enum), **Linux privesc** (DirtyPipe, Dirty COW, Baron Samedit, Docker/LXD group, NFS no_root_squash, PATH hijack, LD_PRELOAD, wildcard injection) and **modern enterprise CVEs** (Spring Boot Actuator, Laravel Ignition, Text4Shell, ColdFusion, PaperCut, TeamCity, OFBiz, vCenter, Cacti, ThinkPHP, WordPress, GraphQL, Zimbra) — each carrying a structured `meta.fingerprint` (`service` + `version_kind` + `versions` + `cve` + `solved_via` + a one-line detection footprint), whose `version_kind` / `versions` shape **mirrors the CVE→exploit index exactly** so the version verdict stays authoritative.

Three properties matter and each is verified:

- **Distilled, not verbatim.** Every entry is a general technique note — the approach, the command pattern, what it leaves in logs — distilled from the offensive- */ bug-bounty methodology and public CVE facts. Not one line is a copyrighted third-party walkthrough. Operator-supplied writeup notes are handled like the phishing lures: imported at runtime with `--operator` from a gitignored engagement directory, marked `operator` , and never committed (entries.jsonl is itself gitignored, so the seed ships only as ingester* code* that regenerates it).
- **Additive, idempotent, consolidation-safe.** The ingester mirrors the recon-methodology template: existing lines pass through as raw bytes, its own lines carry `meta.exploitation_writeup` , and a re-run is **byte-identical** (verified by hash). Each entry is `category="writeup"` + `meta.no_merge` , which `consolidate.py` treats as standalone — so no duplicate is ever created, without re-running the full pipeline (which would risk the downstream-enrichment revert and a Defender quarantine). The file was confirmed present and countable after every write. KB 2,621 → 2,699.

- **Reachable by 2.7, with the version verdict intact.** A fingerprint query for a discovered service returns the seeded writeup **ranked ahead of generic token matches** — verified end-to-end through the real KB search (vsftpd 2.3.4, log4j 2.14.0, gitlab 13.9.0, drupal 7.50, apache 2.4.49 all range-match their writeup at rank 1) — and an **out-of-range** version does not wrongly match (log4j 2.17.0, apache 2.4.58, gitlab 14.0.0 correctly do not). `retrieve()` gained an optional entry-hydrator so the structured `meta.fingerprint` (which the lightweight search index does not carry) is available to the range match; the exact-version substring path remains the fallback.
- **Now wired into the live loop.** 2.7 is no longer only plumbing: `main.py` injects the KB retriever into the orchestrator at startup, and the proposer prompt carries a **FINGERPRINT-MATCHED WRITEUPS** block for the exact service+version fingerprints in state — "this stack is commonly solved via X", with the `solved_via` lead and a citable entry id. Verified live: a `vsftpd 2.3.4` service in state pulls its backdoor writeup into the prompt. Additive and injected: with no retriever wired (the hermetic tests), the block is empty and the prompt is unchanged; it retrieves, and like the rest of the proposer path it runs nothing. The remaining growth is content — more distilled fingerprints — not code.

Verification

- **Hermetic safety suite** (`sh backend/run_safety_tests.sh`) — green after every phase, expanded across all five with `test_phase1_runtime` , `test_state` , `test_scope_hostcheck` , `test_credvault` , `test_corpora` , `test_detection` / `test_detection_safety` (OPSEC channel + blue-view-unchanged), `test_repeater` , `test_tunnels` , `test_report_templates` , persistent-shell containment tests in `test_kali` , Phase 5's `test_terminal` (PTY containment + the sentinel shell provably untouched) and `test_exploits` (version comparison, tiered ranking, executes-nothing), and the Windows backend's `test_winrm` + `test_winrm_safety` (host-locked / no gate bypass / secret never leaks / orchestrator can't auto-run WinRM) with the AD oracle extended to the native Windows variants, plus the post-assessment `test_search_ranking` (substance-gated tier boost + completeness nudge) and `test_codescan_rules` (8-language rule bundle + resolver), plus build #4's six new suites — `test_sliver` / `test_sliver_safety` , `test_obfuscation` / `test_obfuscation_safety` and `test_evasion` / `test_evasion_safety` (no agent path, gated-vs-human-only split preserved, `<listener>` never substituted, the tunnel key never crossing the HTTP boundary, the artifact never executed, and a negative control proving that stripping the OPSEC note makes the engine **refuse** rather than degrade), plus build #5's `test_scans` — the shared source scanner's own regression file, which runs **first** in the suite and plants a violation of each shape (unscanned package, basename-exempted path, aliased import, in-function import, `import_module` string concatenation, f-string) into a temp tree to prove the scanner catches it before ten locks rest on it, and the `I2` follow-on's four new `test_tunnels` cases (a start needs approval + the red-confirm, both server binaries actually trip the heuristic so the danger leg is live, the gated argv is the spawned argv, and the request carries the gate fields defaulting false), plus build #7's `test_prevalidated_gates` — the belt-and-suspenders re-checks on the prevalidated path, proving danger is re-checked in **all three** modes with a windows probe (`Write-Host go ; Invoke-Mimikatz`) that only the whole-script classifier catches, so a re-check wired to the generic heuristic fails rather than passing quietly, carrying positive controls that the same request **with** the ack still runs in all three modes and that a benign request is untouched. plus build #7's continuation — `test_lifecycle_safety` , the lock on the shared listener lifecycle (a watched listener exposes **no stdin writer**, because the child is handed a raw pipe fd rather than `subprocess.PIPE` so `proc.stdin` is `None` ; a whole-tree scan with a planted-violation control keeps the handle inside the four modules that own a listener; every branch of the status derivation is exercised including the one that must never claim a listener, an unprobed port; a dead process is reported dead **with the output it died on**; and a stop is asserted to reap the container-side server targeted at its own argv rather than

the tool in general), the new per-surface cases for the two gated lifecycles (engagement + approval + red-confirm each refusing with nothing spawned, both server binaries in each pair actually tripping the heuristic so the danger leg is live, `-i` present for the console binary and absent for the daemon, and a dead listener refusing rather than reporting up), and the Sliver build's new contract (`generated` vs `failed` decided by the artifact read-back, not the console's exit code, asserted in **both** directions), plus build #8's two new suites — `test_reasoning` (the reasoning copilot's plumbing: the ledger records a failed lead and blocks its re-proposal; the hypothesis/expected_signal/citation schema **fails** an uncited proposal; the frontier persists untried leads and recovers a dead-end without resurrecting a killed one; a connection-refused output yields a reachability diagnostic; the **positive control** that a version-mismatched CVE is caught and downranked while the matching version passes; web/AD/privesc specialist routing; fingerprint retrieval outranking a token match; the model-tier lever inert when unset; and the web-exploit + privesc drafts grounded and cited) and `test_substrate` (the Task-1 verdict function separating a landmine from a version banner from an absent binary with a control, the whole pipeline classified over the **real** catalog with an injected container, and the static Dockerfile coverage) — and the extension of `test_loop` itself onto the whole `reasoning/` package (no execution path anywhere in orchestrator + eleven reasoning modules, by AST call-analysis with a control that a payload *string* is not a false positive, and no auto/batch-approve in the proposer path or frontier), and the exploitation-writeup corpus's retrieval-alignment case (`test_reasoning`): a structured-fingerprint writeup range-matches an in-range version and floats above a token match, while an **out-of-range** version does not match — the version verdict holding through the corpus. Build #9 then added three regression files/cases for the defects its live fire surfaced — `test_secretargs` (credential values redacted out of the persisted record, per-tool, with nmap-ports and password-with- `@` controls), the `impacket-` -prefix parser-name case in `test_state` , the inherited-rights-never-armed case in `test_adorch_safety` , and the `-dc` -needs-FQDN ordering case in `test_adgraph_collector` . **48 test files, 529 checks, 0 failures.**

- **Live-fire proof** (`sh docker/proof/live_fire_proof.sh`) — **66 passed, 0 failed, 3 not-run** against the rebuilt image, after the defects the earlier runs found were fixed and iodine was demonstrated end to end. Passing: an implant that builds through the gated path and a **beacon that calls back**; a **proxychained request routed through the pivot** into an otherwise unreachable subnet, using the argv `wrap_command` produced, with the same tunnel unable to reach `1.1.1.1` ; all three lifecycle gates refusing on all three limbs with nothing spawned; every listener observed bound by `ss` inside the container and still bound at the end of its phase; the pivot subnet entering scope only via the explicit amendment, with the deep target confirmed unreachable first; and containment measured **from inside** the implant host — no internet, no host, C2 only. Also passing: iodine's IP-over-DNS tunnel, through the same gated surface, carrying ICMP with 0% loss and confirmed DNS-encapsulated by a server-side `tcpdump` . Not-run and reported as such: ligolo's interface route (a deliberate won't-do — it needs commands typed into a live C2 console, which HackPit will not do), the dnscat2 client handshake (decisively an upstream defect in the pinned build, above HackPit's proven-bound listener), and a delegated DNS zone (a domain and public listener the operator does not have).
- **The audit's "probed and holds" list, re-run after build #5** — all 41 items still hold, including the `.exe` spellings from `I1` , the four gate orders, the no-silent-downgrade-to-lab path, and Critical 1's eight catalogued tools. The five demonstrated Critical 2 bypasses are all refused. The planted `from cockpit.kali import run_kali` in `adgraph/orchestrator.py` now **fails** the scan (verified against a temp mirror; the repo was never modified). Read-only enumeration — `nmap` , `sqlmap --dbs` , `netexec --shares` , `ldapsearch -x` , `hashcat` , `nuclei` — stays clean on both paths.
- **Docker proofs** — `isolation_proof.sh` 4/4 (lab still cannot reach internet or host), `engage_open_proof.sh` 4/4 (engage has full reach). Both re-verified in build #7 **after** the sandbox stack was rebuilt and recreated from the current Dockerfile — twice, the second time after the `proxychains` fix was added to that Dockerfile — which is a real change to the running containers rather than a no-op re-run.

- **Image build** — `hackpit-kali:build7b` builds from the current `Dockerfile.sandbox` with exit 0 and carries the `proxychains` fix baked in (`socks5 127.0.0.1 1080` , zero `socks4` lines); the stack was rebuilt and recreated from it, and the live fire re-run against that image reports the config as coming from the image rather than from its own runtime patch. `hackpit-kali:build7` builds from the pinned `Dockerfile.sandbox` with exit 0, and every pinned tool resolves and smoke-tests by its real invoked name (`dnscat2-client` , `dnscat2-server` via its four gems, `donut`), with the shipped dnscat2 `HEAD` , gem versions and `donut-shellcode` version confirmed **inside** the image to equal the pins.
- **Browser** — every UI surface exercised against a live Ollama backend per the testing rule: the Phase-4 surfaces (payload-set arsenal rows, the OPSEC red-team channel, repeater send/replay/diff, tunnels route preview, exam report templates with the proof table) and the Phase-5 ones — `:terminal` running `top` and `vim` with live resize, `:exploits` resolving `vsftpd 2.3.4` to the backdoor exploit and its CVE, the state panel's per-service jump into it, and `/category/cloud` at 535 entries. **Windows targets** (`/windows`): profile create/list/test/delete and the AD-walk "run on" picker verified against the backend. As of build #9 the connectivity **test** and a live WinRM round-trip are no longer deferred — they are **demonstrated live** against a real DC through the same HTTP API the UI drives (see the build #9 live-fire below); the browser render of `/windows` and `/cockpit` was confirmed serving 200 while that run executed.
- **Live fire against a real Windows/AD domain** (`backend/livefire_windows.py` → `backend/livefire.log` , build #9) — **45 passed, 0 failed, 3 not-run** against the operator's VMware Server-2022 `corp.local` DC, driving the live backend over its real HTTP API. Demonstrated live: a WinRM round-trip landing on the profile host with the credential absent from every record, the whole-script danger classifier firing on real input, real `bloodhound-python` collection parsing into the typed graph (84 nodes / 625 edges) with the DA path computed on it, a real DCSync returning `krbtgt` through the red-confirm with loot ingesting into state, and the walk advancing only on the approved exit-0 run. It also found and fixed four defects invisible to the hermetic suite (credential-in-record leak, impacket parser-name mismatch, KB grounder arming a free edge, collector FQDN classifier), and truth-checked the detection catalog against the DC's own Event Log — confirming DCSync's 4662/DS-Replication signal (12 of 12) and **correcting two catalog claims the real telemetry contradicted** (secretsdump's mode-specific artefacts, and `ad_collect` raising no 4662). Not-run and reported as such: a Windows-target C2 callback (needs a routable listener), the native `Invoke-Mimikatz` DCSync variant (no offensive tooling on a bare DC), and a model-spontaneous DCSync pick.
- **Frontend** — `tsc` clean, lint at the pre-existing baseline (11 errors + 1 warning, unchanged — verified by stashing the changes and re-running), `next build` exit 0 (routes include `/terminal` , `/exploits` , `/windows`).
- **The launcher, verified against real state rather than a mock** (build #11) — the status rail was exercised in both directions on the same page with no code change: with Docker down it reported `sandbox stack down` with **7 tiles dimmed and 7** `stack down` **markers**; after `docker compose up -d` it reported `up · engage up` with **0 dimmed and 7** `needs the stack` . The freshly created networks were re-checked against the isolation floor — `assert_isolation_proven()` passes and `hackpit-isolated` is `internal=true` while `hackpit-open` and `hackpit-engage` are deliberately not. 4 bands, 15 surface tiles and 30 category cards render live.
- **Operator identity, checked against git itself** (build #11) — `test_operator.py` asserts `backend/operator.json` is ignored by asking `git check-ignore` rather than by reading the ignore file and trusting the pattern, and carries a control proving the predicate can answer "no" for a tracked file. The staged diff was grepped for the real name and identifiers before committing; only synthetic fixtures remained.

Status

Build #4 (AV/EDR evasion + traffic obfuscation) is complete and verified: image `hackpit-kali:build4` builds with all eight new binaries smoke-tested by invocation, the full hermetic suite was green at 42 files / 446 checks at that point, and the frontend builds clean. Tasks 8–13 have now had their independent review (see Part II) — no containment hole, five substantive defects fixed, each with a regression test. **Two caveats are carried forward as open items in Part II rather than papered over**, and build #4 should not be called field-ready until they close: the C2 and tunnel surfaces have never been run against a live beacon, a real delegated DNS zone, or an instrumented Windows host, so their *efficacy* claims are documented rather than demonstrated (the *containment* claims are what the suite locks, and those hold without live infrastructure); and three installs in the image layer are still unpinned, so that layer is not yet byte-reproducible.

A note the review sharpened, because it cuts against the instinct to trust the tooling: `pipeline/detection_sources.py --verify` reported **0 problems** throughout, including while `evasion_etw_blind` carried a technique id about shell history. It verifies that every id, name, tactic, log source and Sigma reference matches live upstream — it cannot verify that the technique chosen is the right one for the activity being described. A clean `--verify` is a check on transcription, not on judgement.

Build #5 (gate integrity) is complete, and the audit's one remaining important item is closed with it. All three gate-audit criticals are now closed: Critical 1 in build #4's fix wave (`4492fec`), and Criticals 2 and 3 here. `I2` — the ungated tunnel listener start, outside the plan's scope — was recorded open, then fixed as a follow-on in the same build. The suite stands at **43 files / 477 checks / 0 failures**, the frontend builds clean, and the audit's full "probed and holds" list was re-run with nothing regressed. What changed is not only three guards but the conditions under which a guard is allowed to be believed: `backend/AGENTS.md` now requires a safety test to iterate real data, assert on what it actually checked, and carry a demonstration that it can fail — and `test_scans.py` runs first in the suite to hold the shared scanner to that standard before ten locks depend on it. The remaining work is the deferred list in Part II — chiefly live-fire verification of the C2 and tunnel surfaces, and the three unpinned installs in the image layer.

Build #7 (live fire, image pinning, prevalidated danger re-check) is complete, and it is the first build whose headline result was a set of defects rather than a set of features. Two of its three tasks closed cleanly: the image layer is byte-reproducible and `hackpit-kali:build7` builds green, and the prevalidated path now re-checks danger in all three modes. The third — live-fire verification, Part II's largest outstanding item — did what it was supposed to do. It proved the gates fire against real binaries and proved containment holds live from inside the contained host. It also found that **three of the surfaces did not work as shipped**: the Sliver implant argv named a subcommand that does not exist, and both listener lifecycles reported `status="listening"` for a process that had already exited. None was a containment failure — nothing escaped and no gate was skipped — but they meant the efficacy claims those surfaces carried were never true. Two of the three were found only because a real build and a real process replaced a structural assertion; the pinning task independently surfaced a smoke test that had never passed for the same reason.

All three are now fixed, and the design question the run left open is decided (2026-07-28). The implant builds through the console `sliver-server` hosts and its success is decided by reading the artifact back rather than by a console exit code; both listeners hold a console binary's stdin open — as a raw fd with no writer object, so the surface gains liveness without gaining the ability to type into a live C2 console — and report a status they observed, with a dead process now a refusal rather than a fiction. The live fire stands at **66 passed, 0 failed, 3 not-run**: a beacon calls back, a proxychained request routes into an otherwise unreachable subnet, iodine's IP-over-DNS tunnel carries traffic confirmed DNS-encapsulated on the wire, and every gate still refuses on every limb. The stale precedent is resolved by tightening rather than by deleting the citation: the DNS-tunnel listener is gated exactly as `I2` gated the pivot listener, because a covert channel *is* an exfil route; the Sliver server is gated too, on

the narrower ground that its config can persist listener jobs that come up with the daemon, so "starting it is inert" is a property of a config file rather than of this code. Human-only is untouched and remains the load-bearing property; the gates are belt to those suspenders.

Fixing them surfaced two more defects of the same family, both also fixed: a stop that killed the `docker exec` client while the server kept running and holding its port inside the container, and a `proxychains` config shipped pointing at Tor rather than at the SOCKS5 port HackPit's own rewrite targets — a shipped path that every unit test passed and that could not have worked. A second pass then verified both of those against a rebuilt image rather than against a diff — the `proxychains` layer now builds and the config is baked in, and the stop-reap was demonstrated live against the failure that motivated it, including that reaping one tunnel leaves a neighbouring one bound. That pass surfaced a sixth defect of the same family: a status observed once and never re-observed upward, which left a listener that bound just after its settle window permanently `starting` and therefore permanently unroutable. It now refreshes in both directions, and only on evidence. That is the durable lesson of this build, and it is worth stating plainly: **six defects in these surfaces were invisible to a green test suite, and every one of them was a claim asserted structurally that nothing had ever executed.** The suite now stands at **45 files, 502 checks, 0 failures**, with `test_lifecycle_safety` locking the new shared lifecycle — including the positive controls that a dead process is reported dead and that an unprobed port is never called listening. Two of the four not-runs then closed: iodine's DNS tunnel is demonstrated end to end, and dnscat2's failure is pinned decisively to the upstream build rather than to anything HackPit owns. What remains genuinely undemonstrated is stated as such and not folded into a pass: a delegated DNS zone (an operator prerequisite), ligolo's console-driven interface route (a deliberate won't-do), and detonation on an instrumented Windows host.

Build #8 (the reasoning copilot + substrate proof) is complete. It did the two things it set out to do without moving the safety boundary an inch. The substrate claim is now a *measured* one — **97 of 104 catalogued Linux tools actually execute** in the live image under its real profile (93.3%), with the seven that do not reported honestly as two `sudo`-wrapper landmines and five genuine absences, and the probe surfaced two real catalog defects (Sliver/Empire installing under a different binary name) that are now fixed and forced through the danger gate under their real invoked names. The orchestrator is a genuinely deeper proposer — working memory that stops it re-proposing dead leads, hypothesis-first proposals with an enforced citation gate, a scored candidate frontier with dead-end recovery, failure diagnosis, a refute-first critic that kills version-mismatched CVE hallucination, specialist routing, fingerprint retrieval, and a model-tier config lever — and **the honest ceiling is stated plainly: reasoning depth is mostly the base model, so this moves the loop up the curve on easy and known-pattern-hard, not to "solves anything."** The one line that did not move is the one that matters: **the loop reasons but never executes, and approval stays per-command** (D18), re-locked by extending the source-scan onto the whole `reasoning/` package and by a scan proving no auto/batch-approve exists in the proposer path or the frontier. The suite stands at **47 files, 519 checks, 0 failures**; the loop was validated live against Ollama (proposals cited, reacting to prior output, not re-looping, every command still stopping for human approval).

The follow-on the build flagged is now **landed**: the **KB exploitation-writeup corpus** that 2.7's fingerprint retrieval keys into. `pipeline/ingest_exploitation_writeups.py` seeds 78 distilled, non-verbatim technique writeups keyed by service+version fingerprint (the CVE-index `version_kind` / `versions` shape, so the version verdict stays authoritative), additively and idempotently (byte-identical re-run, `category="writeup"` / `no_merge` so consolidate never duplicates them, file confirmed un-quarantined; KB 2,621 → 2,699). Operator writeups import at runtime into gitignored engagement data and never reach the repo. Retrieval alignment is verified end-to-end through the real KB search: an in-range version returns the seeded writeup ranked ahead of token matches, and an out-of-range version does not wrongly match. The corpus is intentionally a seed — growing it further stays an ongoing content task, not a code one.

Build #9 (the Windows/AD path, driven live against a real domain) is complete. It is the build that moved the Windows/AD path from "built and locked hermetically" to "demonstrated live," and — like build #7 before it — its lasting value was as much in what a real target exposed as in what it confirmed. It confirmed a great deal: a WinRM round-trip locked to the profile host, the whole-script danger classifier firing on real input, real BloodHound collection parsing into the typed graph, and a real DCSync returning `krbtgt` through the red-confirm with loot ingesting into state — the gates holding at every step, on real input, 45 passed / 0 failed / 3 not-run. It also found **four defects the green hermetic suite could not see, every one a place where two halves of the codebase agreed in a test and disagreed against a real domain**: a domain credential persisted into the run record (and thence into the LLM proposer context), impacket loot ingesting as nothing because the catalog and the parser spelled the tool differently, the KB grounder arming an inherited-rights edge with a destructive command, and a failure classifier that misdiagnosed the collector's FQDN error. All four are fixed and regression-locked; the suite stands at **48 files / 529 checks / 0 failures**. Task 5 was the first time the detection describe-side was checked against real telemetry rather than documentation — it confirmed DCSync's high-fidelity 4662 signal and corrected two catalog claims the DC's own Event Log contradicted. What stays not-run is stated plainly and not folded into the pass: a full Windows-target C2 callback (a routable listener the sandbox does not expose by choice), and the native-tooling abuse variants (offensive tooling on a bare DC, out of scope). The durable lesson repeats build #7's in a new domain: **a live target is the only thing that finds the bugs that live between two components that were each tested alone.**

Build #10 (the opt-in C2 exposure override, and an honest NOT-RUN) — complete. Build #9's Task 4 recorded a Windows-target C2 callback as NOT-RUN for one concrete reason: the Sliver/tunnel listener lives in `hackpit-engage-sandbox`, which by standing invariant publishes no ports, so the lab DC had no route to it. Build #10 supplies exactly that missing route **without widening the default posture**: `docker/proof/c2-lab.yml`, an opt-in compose override that publishes a single port — `192.168.13.1:53/udp`, the iodine tunnel server's DNS port bound to the one host interface (VMnet8) the lab DC can reach — and nothing else, never a wildcard. It is never applied by the documented bring-up; `backend/test_exposure_safety.py` locks three invariants (default compose publishes nothing, every override port names a literal host IP, nothing composes it in implicitly) and plants a violation to prove the scan can fail. The override is not only built but **demonstrated live** — the exposure came up on `192.168.13.1:53/udp` and tore down cleanly. On it sits a gated C2/AD proof harness (`docker/proof/c2_0{1..4}_*.sh` + `c2_lab_proof.sh` orchestrator + in-process driver) built on build #7's discipline: offensive command strings are never inlined — they are operator-supplied paste values read by file path, and an empty value reports **NOT-RUN**, never a fake pass. The four offensive proofs it drives — staging the tunnel client on the DC, the iodine DNS tunnel, a Sliver beacon over it, and a native DCSync behind a scoped Defender **exclusion** (not a real-time-protection toggle) removed in a `trap` guard — are **NOT-RUN**: harness-ready but requiring operator-supplied commands that were deliberately not filled, recorded as such rather than folded into a pass. A final hardening pass closed the build's engineering items and corrected one of its own claims: the suspected `run_safety_tests.sh` gating hole did **not** exist (`set -e` has been in the committed runner all along, and a planted failure was verified to abort it), while the failing `nc` danger-gate test turned out to be an environment leak rather than an over-block — the one test in `test_session.py` that called the gates outside the hermetic fixture, so it reached a live Docker probe. Both are now fixed and the runner additionally checks every test's exit code explicitly. The build also added a read-only DC prerequisite probe and a harness-honesty regression test, and that test immediately caught a real defect: three of the four proof scripts' `[[PASTE]]` slots were not actually empty, so an unfilled proof was one WinRM round-trip away from being scored as a genuine attempt. Suite: **538 checks across 50 files, green**. The four offensive proofs remain **NOT-RUN**.

Build #9 — the Windows/AD path, driven live against a real domain (2026-07-31)

Everything in HackPit's Windows/AD path was built and locked **hermetically**: `test_winrm` / `test_winrm_safety` monkeypatch the WinRM transport, and the AD graph, orchestrator and technique catalog were all exercised against `adgraph/sample_data.py` — a GOAD-shaped synthetic domain. Nothing had ever touched a real domain. Build #9 stands up the operator's own lab (VMware Windows Server 2022, promoted to a `corp.local` forest, WinRM over HTTP:5985, `Administrator@CORP`) and drives the **live** backend over its real HTTP API against it. It adds no new capability and no autonomy: every live command clears the existing gates and the human approves each one; the orchestrator proposes, it never fires. The evidence is `backend/livefire.log`, produced by `backend/livefire_windows.py` — a by-hand harness that is deliberately **not** part of the hermetic suite (which stays runnable with no VM, no network and no `pywinrm`). It records every check as PASS / FAIL / NOT-RUN, and a check that could not run is NOT-RUN, never a silent pass — the whole point being that "demonstrated live" and "still needs X" are different claims. **45 passed, 0 failed, 3 not-run.**

Task 1 — the WinRM round-trip and the gates on real input (14/14). A real PowerShell command runs on the DC over WinRM and comes back with the box's own identity (`WIN-990RALNGERV`, `corp\administrator`, `corp.local`) — the proof the host-lock held, since the request carries no host field and the destination is resolved server-side from the profile id. An unapproved command is refused live at the approval gate; a foreign host named in the args cannot redirect the run (it is echoed *by* the DC, which is where the run still landed); and build #5's whole-script danger classifier fires **on real input** against `Write-Host go ; Invoke-Mimikatz` — the exact argv[0]-vs-whole-script case that motivated it — then clears once the human acks. The credential appears in none of the run record, the event stream, or the recorded command line; the run is audited and retrievable by id; and real WinRM stdout ingests into state (an OSCP-style proof flag attributed to the profile host). This is the live half of the WinRM deferral, closed.

Task 2 — the AD graph off real collection (10/10), not `sample_data`. The gated, approved, scope-locked `bloodhound-python` collects the **real** `corp.local` over the engagement executor — 84 nodes, 625 edges, the live DC's own computer object present — parses into the typed graph, and the route to Domain Admin is computed on that real graph. The collector preview redacts the credential; an unapproved collection is refused live at the approval gate; and an **off-scope DC is refused live at the target gate**. This closes both "AD off synthetic data" and "touch a real AD domain."

Task 3 — the live abuse walk (12/12, 2 not-run). The orchestrator proposes edges off the real graph; the human approves; the executor runs them. A real **DCSync** (`impacket-secretsdump -just-dc`) executes against the domain and returns real credential material — four NTLM hash lines including `krbtgt` — after the danger red-confirm is demanded and supplied; the dumped credentials ingest into state (`administrator`, `krbtgt`, `win-990ralngerv$`); and the walk advances **only** on that approved, exit-0 run (advancing a commanded edge with no run is refused). A native Windows AD action (`Get-ADGroupMember 'Domain Admins'`) runs on the DC over WinRM. Two edges are recorded NOT-RUN, honestly: the model happened to pick `GenericWrite` over DCSync on the run, so the destructive step was operator-directed rather than model-selected; and the native `Invoke-Mimikatz` DCSync variant cannot run because a freshly promoted Server 2022 carries no offensive tooling, and staging some onto the DC was out of scope — the Linux/impacket variant is what executed.

Four defects the live run found that the hermetic suite could not — all fixed, each now regression-locked. Every one was a claim that two halves of the codebase agreed on in a test but disagreed on against a real domain:

- **The domain credential leaked into the persisted record.** The WinRM path never puts a secret on a command line — the profile resolves it server-side — but every credentialed Linux tool (`bloodhound-python -p …`, `impacket-secretsdump DOM/user:pass@host`) carries it as an argv token, which was stored verbatim in the run record, and from there into rendered reports **and the LLM proposer context**. So collecting a domain shipped

the DA password to the model. `cockpit/secretargs.py` now redacts credential *values* out of the argv at the record boundary (per-tool, never a blanket flag list — `-p` is a password to netexec and a *port list* to nmap), wired into both `RunRecord` constructions. A first cut split the impacket positional on the *first* `@` and leaked a password fragment out the "host" side; the fix anchors the host to the last `@`. `test_secretargs.py` locks both the positive cases and the negative controls (nmap ports survive untouched), including a password-with-`@` fragment check.

- **Real DCSync loot ingested nothing.** The AD technique catalog proposes Kali's `impacket-secretsdump`; the state parser registry was keyed only on upstream's `secretsdump` / `secretsdump.py`. Two halves disagreeing about one tool's name meant four dumped hashes ingested as zero credentials. `state/ingest.py::program_name` now strips the `impacket-` prefix; `test_state.py` locks that every spelling reaches the same parser (and that `impacket-wmiexec` does *not* become a secretsdump parser).
- **The KB grounder armed a "free" edge with a destructive command.** `MemberOf` and `HasSIDHistory` are inherited rights — nothing to run — but their KB seeds still matched a real ACL-abuse entry, whose example commands were adopted wholesale. On the real graph the orchestrator therefore proposed a runnable `net rpc password` (a destructive reset) for `ADMINISTRATOR -MemberOf-> DOMAIN ADMINS`, an edge with no action, rendered `resolution="ready"`. The gates all held — nothing fired — so this is a correctness lock, not a containment one, but a panel that asks a human to approve a domain change for a free step is how a hurried operator makes an unintended one. A `no_command` flag on the spec keeps the KB *citation* while refusing to let it manufacture an *action*; `test_adorch_safety.py` locks it with an adversarial grounder that always offers a destructive command, plus a positive control that an actionable edge is still groundable.
- **The collector's failure classifier had nothing to say about the FQDN error.** `bloodhound-python` refuses an IP for `-dc` ("looks like an IP address, but requires a hostname (FQDN)"), and that same message also mentions a "DNS server IP", so the generic DNS signature matched it and told the operator to fix their nameserver when `-dc` was the real problem. A signature for it now runs **first**; `test_adgraph_collector.py` locks the ordering.

Task 4 — Windows-target C2 (feasibility, measured; not run). Build #7 demonstrated Sliver + iodine against Linux. Completing that against this Windows target needs a callback path from the DC to the listener, and the harness **measures** rather than assumes one: the DC reaches the host's VMnet8 gateway and has DNS egress, but the Sliver listener lives in `hackpit-engage-sandbox`, which publishes no ports and sits on a Docker bridge the DC has no route to. Recorded NOT-RUN with the exact unblock (publish the listener port from the sandbox to the host, point the implant at the gateway) and the reason it was not done here: publishing a port changes the sandbox's network posture, and this build's standing invariant is that it adds no new exposure.

Task 5 — the detection footprint, truth-checked against real telemetry (8/8) — the purple-team validation the describe-side had never had. The catalog's footprint claims were written from documentation; this reads the DC's own Event Log back after techniques that really ran. The audit policy is read first (so "not observed" is distinguished from "never audited"), and events are counted as **before/after deltas keyed on** `EventRecordID` — the VM's clock drifts and w32time hauls it back, so a timestamp window silently lands in the future and reads as "the box logged nothing" (an earlier iteration of the harness "disproved" claims that were in fact correct, exactly this way; the log also flushes asynchronously, so the query settles on the DC first). Confirmed against real telemetry: DCSync raises Security **4662 with the DS-Replication GUIDs** — 12 of 12 events carried them — under the DC's **default** audit policy, so "loud" is the right rating; and T1003.006 is the right id. **Two catalog claims the real box contradicted, now corrected (atomic with `attck.py`, per the detection trap):** (1) the `secretsdump` telemetry list lumped SAM/LSA-mode artefacts (5145 admin-share access, 7045 short-lived service, hive saves) together with the `-just-dc` path — a live `-just-dc` run added **zero** of them, because it replicates over DRSUAPI and touches no share, service or hive; the entry now says which mode each artefact belongs to. (2) the `ad_collect` entry promised "Security 4662 … if object auditing is enabled" — but on a DC that auditing is on by default, and a full `-c All` collection added **zero** 4662 (4662 needs a SACL on the objects read; LDAP enumeration

produces network logons, not directory-access events); the entry now points at LDAP diagnostics (1644) as the dependable host signal, while the "loud" rating stands on query volume. `test_detection` / `test_detection_safety` stay green after both edits.

The honest bottom line: the WinRM round-trip, real AD collection, the real DA path and the executed DCSync abuse are **demonstrated live**, with the gates holding on real input; a full Windows-target C2 callback and the native-tooling abuse variants still need more lab infrastructure (a routable listener, offensive tooling staged on the DC), and are recorded as not-run, not as passed.

Build #10 — the opt-in C2 exposure, and an honest NOT-RUN (2026-07-31)

Build #9 closed the Windows/AD path live but left one item explicitly NOT-RUN: a full Windows-target C2 callback. The reason was precise and worth not papering over — the Sliver/tunnel listener runs inside `hackpit-engage-sandbox`, which by standing invariant publishes no ports, so the lab DC (on VMware's NAT subnet) had no route to it. Closing that needs a change to the lab's exposure posture, and build #9's rule was "no new exposure by default." Build #10 supplies the missing route as an **opt-in** and does the surrounding engineering; it does **not** fire the offensive proofs, and says so plainly.

The exposure override — built, locked, and demonstrated live. `docker/proof/c2-lab.yml` is the one place HackPit publishes a host port, and it is never applied by the documented bring-up — reaching it takes a second, explicit `-f docker/proof/c2-lab.yml`. It publishes exactly one port, `192.168.13.1:53/udp`: the iodine tunnel server's DNS port, bound to the single host interface the lab DC can see (the VMnet8 address), never a wildcard that would also expose it to whatever network the laptop is on. `backend/test_exposure_safety.py` locks three invariants hermetically — (1) the default `docker/docker-compose.yml` publishes zero ports; (2) if the override exists, every published port names a literal host IP and never `0.0.0.0` / `::` / `*`; (3) nothing else in the repo composes the override in implicitly — and it plants a wildcard-bound port and proves the scan catches it, so a silently-broken scanner cannot make the checks vacuous. The mechanism is not only tested but **demonstrated live**: the exposure came up on `192.168.13.1:53/udp` and tore down cleanly, leaving no published port behind.

The proof harness — the same honesty discipline as build #7's live fire. On top of the override sits a gated C2/AD proof harness: four scripts (`c2_01_dc_prereq_stage.sh` staging the tunnel client + TAP driver on the DC, `c2_02_iodine_tunnel.sh` the DNS tunnel, `c2_03_sliver_beacon.sh` a beacon over it, and `c2_04_dcsync_defender_excl.sh` a native DCSync behind a **scoped Defender exclusion** — `Add-MpPreference -ExclusionPath` for the tool path, removed in a `trap` guard, *not* a real-time-protection toggle), plus an orchestrator (`c2_lab_proof.sh`, the only script permitted to compose the override up and down) and an in-process driver that reuses build #9's credential redaction. The load-bearing property carried over from build #4/#7: **offensive command strings are never inlined in the harness.** Each is an operator-supplied paste value handed to the driver by file path; an empty or unfilled value makes the proof report **NOT-RUN**, never a fake pass. Every offensive step stays behind the existing engagement + approve-each + danger red-confirm gates; no new autonomous path is added.

What is NOT-RUN, and why it is recorded as such. The four offensive proofs were **not fired**. They are harness-ready but need operator-supplied commands (the iodine client invocation, the Sliver implant generation and launch, the DCSync one-liner), and those were deliberately not filled — so the run reports NOT-RUN across all four, which is the honest state, not a failure. "The harness is built and safe" and "the offensive path was demonstrated end to end" are different claims, and only the first is made here.

The safety harness, investigated — and one of this build's own claims withdrawn. Wiring the new exposure test in raised a suspicion that the runner itself was not gating: `backend/run_safety_tests.sh` appeared to invoke every test file without checking any exit code, which would mean a failing safety test could let the suite exit 0 and print "passed." **That claim was wrong, and it is worth recording as wrong rather than quietly dropping.** `set -e` is present in the committed runner and has been since before this build; planting a deliberate failure and running the suite produced exit 1, a halt at the second test file, and no "passed" banner. The suite always gated. What *is* a real weakness is that `set -e` is silently suspended for any command inside a pipeline, an `if` / `while` condition, or an `&&` / `||` chain — so a later edit as innocent as `"$PY" test_x.py | tee log` would disarm the whole suite with no visible change in output. The runner now routes every test through a `run_test()` helper that checks the exit code explicitly, names the failing file and the invariants it guards, states how many files had passed before the abort, and reports the file count on success. A guard that can be switched off by accident is not one a safety suite should rest on, even when it happens to be working.

The `nc` danger-gate failure: an environment leak, not an over-block. `test_session.py`'s `test_start_trips_the_danger_heuristic` failed on unmodified code with "nc must start once explicitly acked" — `validate_start` refusing `nc -lvnp 4444` even *with* `dangerous_ack`. The danger/target/scope logic turned out to be correct and the engagement/Wall-A state clean (the request resolved to `lab` mode, no active engagement). The real cause is the LAB gate order — target → approval → danger → **isolation** — where that last gate calls the live `assert_isolation_proven()` Docker probe. The *un-acked* half of each assertion pair short-circuits at `danger` and never reaches it; the *acked* half falls all the way through, so on a host with the stack down it came back `gate="sandbox"` and the test failed for a reason with nothing to do with the heuristic it guards. It was the only test in the file that called the gates outside the `_Spy()` fixture that stubs that probe — contradicting the module's own "Hermetic: … no Docker daemon" contract. Fixed by running it inside the fixture like every other test in the file, with the gate order written into the docstring so the next person does not re-learn it from a red suite.

Harness honesty, locked — and a real defect caught doing it. `backend/test_proof_honesty.py` locks the property the whole NOT-RUN story rests on: an unfilled offensive slot can never become a fake pass. It checks four independent angles — the reader treats empty/whitespace/comment-only as unfilled; driving the real driver subcommands with an empty slot emits NOTRUN, emits no PASS, and never reaches the WinRM transport (asserted with a tripwire, not assumed); an AST pass proves every `_read_paste` call site is followed by a NOTRUN empty-guard, so a *new* slot added later without one fails the suite; and the slots in the four committed scripts really are unfilled — plus a control that feeds it a filled slot to prove the checks can fail. Writing it caught a genuine defect: `c2_01`'s two slots and `c2_04`'s DCSync slot did **not** ship empty. They opened with a self-referential shell fragment (`echo $(cat …| grep …| cut …)`) left from an earlier placeholder sweep, sitting *outside* the first pair of quotes. That text is not a comment, so `_read_paste` returned it as live content, the empty-guard never fired, and the driver would have sent it to the domain controller as PowerShell and scored its exit code as a real attempt — an unfilled proof reporting FAIL, or conceivably PASS, instead of NOT-RUN. The slots are now comment-only and the regression is locked by a test verified to fail when it is re-planted.

The DC prerequisite probe, finalised. `docker/proof/dc_prereq_probe.py` answers whether the C2 path even has the pieces it needs — adapter inventory, staged tooling, DNS/HTTPS egress, route to the VMnet8 gateway, Defender posture, OS/PowerShell version — before anyone runs a proof. It is read-only and *structurally* so: every probe is checked against a read-only cmdlet allow-list that **fails closed**, so a probe added later with a mutating verb makes the file refuse instead of run. The stale hardcoded profile id was removed (`--profile` is now required, so it can never fire at whatever host an old id happens to name) and a `--list` mode prints the probes while contacting nothing. It was **not** run against the DC in this session.

Suite state. `sh backend/run_safety_tests.sh` exits 0 with **538 checks across 50 test files** — now genuinely gated, verified by planting a failure and confirming the suite goes non-zero, names the offending test, and prints no "passed" banner.

Build #11 — the launcher, and operator identity (2026-07-31)

Two small builds, both closing gaps that had been visible for a while and neither moving the safety boundary.

A dozen surfaces were built and then invisible

The top nav carries five product sections and deliberately stays that size. Everything else — `:arsenal`, `:c2`, `:tunnels`, `:windows`, `:repeater`, `:scripts`, `:code-scan`, the AD graph — was reachable only by typing the URL. This was not a new discovery: the project's own working notes had flagged it twice (2026-07-26 and 2026-07-27) without it being fixed.

The launcher is 15 tiles in four bands — **plan**, **operate**, **infrastructure**, **reference** — merged into the home page per D19. The band label is not decoration: it states the posture of everything under it ("*proposes · never executes*", "*every command human-approved*", "*gated start · human-only stdin*"), so the page cannot show a shell tile without also saying who approves it.

`CategoryGrid` is now categories only. It had been carrying **five** product cards — attack-paths, Cockpit, the scripts arsenal, the tool arsenal and code scan — every one of which is now a launcher tile; keeping them would have shown each surface twice on one screen. `ScriptsCard`, `FEATURED` and `COCKPIT_FEATURE` are left in the tree unreferenced rather than deleted, so restoring the old layout is a revert of one file.

The status rail answers a question the UI could not

Every execution surface shells out to `docker exec` and refuses with "*bring the stack up*" when the container is down. That state was only discoverable by clicking into a surface and reading an error. `GET /home-summary` now reports it up front — stack, LLM model, WinRM target, active engagement — and the tiles that need the stack dim when it is down.

Two details that are the point rather than polish. A **null** probe renders as "*unknown*", never as "*down*": an undeterminable probe must not send the operator chasing a container problem they do not have. And the endpoint is deliberately **separate from** `/stats`, because it runs docker probes and folding it in would gate the hero's counters behind a container inspect.

It lives in `main.py` because it is cross-cutting by construction — sandbox probe, LLM config, Windows profile store, engagement store, KB counters — and `cockpit` and `arsenal` may not reference each other.

The report could not identify its own author

`backend/report.py` had **no author, candidate or OSID field anywhere** — every "author" match in it was the word "*authoritative*". An OSCP submission is not attributable to a candidate without a name and an OSID, so the report this tool generates could not be submitted without hand-editing identification in. For a project whose tagline is "*pass the cert*", that is a defect, not a missing nicety.

Fixed per D20, with the block spliced under the report title the same way evidence and the CVSS block are.

A mistake worth recording

The identity module was first written as `backend/operator.py`. `backend/` is first on `sys.path`, so that filename **shadows the stdlib `operator` module for every import in the process**. It was caught immediately and renamed to `operator_identity.py`; `test_operator.py` now asserts `backend/operator.py` does not exist and that `import operator` still resolves outside the repo. It is recorded here because the failure mode is silent, process-wide, and would have been very hard to attribute later.

What is locked

`test_home_summary.py` and `test_operator.py`, both in the suite. Between them: no secret reaches the browser (checked against the **real** stores — stored WinRM secrets, the configured LLM key, every credential in state — and reporting per-source counts so an empty leg is *visible* rather than absorbed into a reassuring total); the endpoints call only the masked accessors, asserted over the **AST** rather than a substring; a status endpoint cannot reach an execution path; the operator config is gitignored, asserted by asking git; and the OSID and email never reach the browser. Every check carries a positive control, and the leak test asserts it cannot go vacuous.

Suite state. `sh backend/run_safety_tests.sh` exits 0 across **52 test files** (50 → 51 → 52). Frontend lint holds at the pre-existing baseline of 11 errors + 1 warning; new tiles stagger via CSS `animation-delay` specifically so as not to add more `react-hooks/set-state-in-effect` errors, and `next build` exits 0.

Build #11 (the launcher + operator identity) — complete. Neither half moved the safety boundary; both closed a gap that had been visible and unaddressed. The launcher's value showed up immediately and by accident: bringing the Docker stack up during verification flipped the rail from `down` to `up · engage up` and un-dimmed seven tiles with no code change, which is the whole feature demonstrated end to end. The report identity block closes a real submission blocker — the generated OSCP report previously carried no candidate identification at all. One near-miss is recorded in full above (a module named `operator.py` shadowing the stdlib) because the failure would have been silent and process-wide.